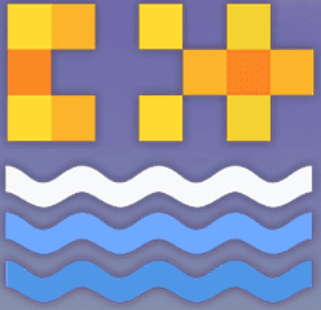




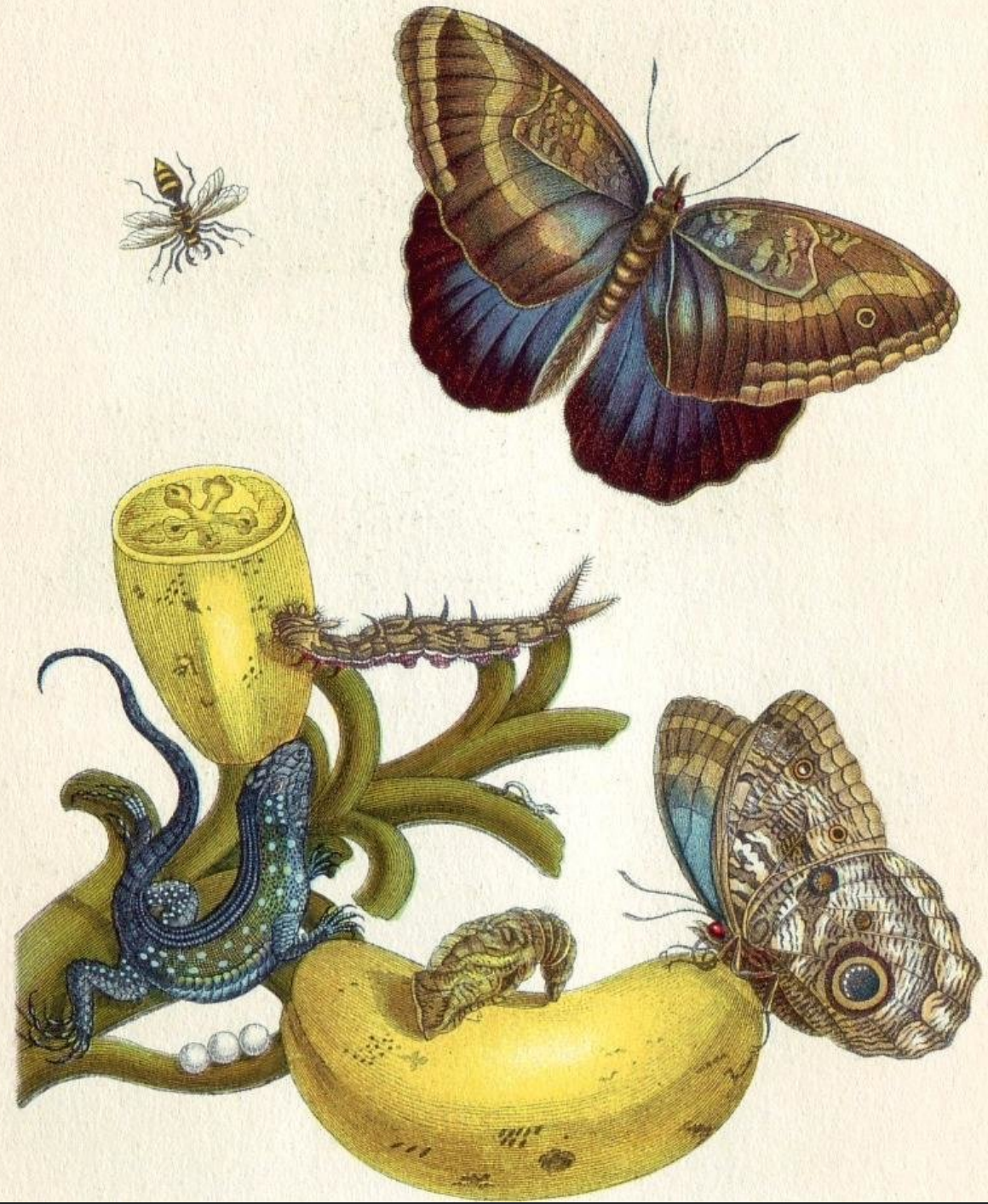
Keynote: **Not Getting Lost in Translations**

Daniela Engert

2024

Outline

- The beginnings
- Let's talk
- A new library
- A new world
- What now
- An experiment
- Move on





ABOUT ME

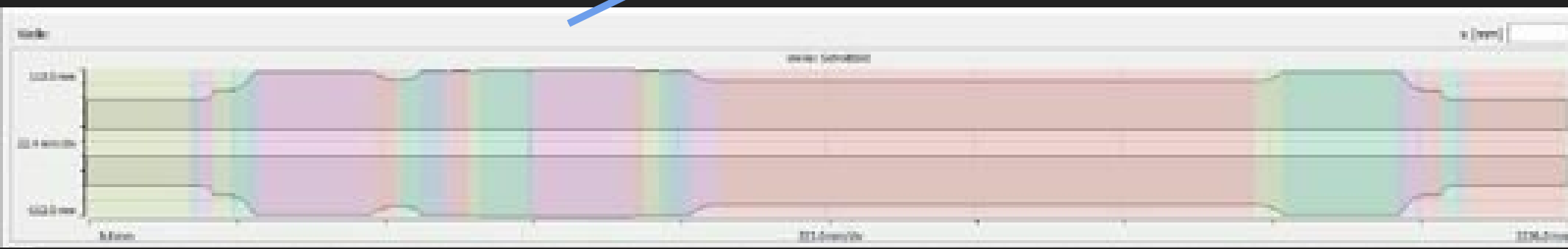
- Electrical engineer
- I build computers and create software for more than 40 years
- I develop hardware and software in the field of applied digital signal processing for more than 30 years
- I'm a member of the C++ committee (learning novice) for 4 years (EWG, SG15)













←
END





TIMELINE

2009



TIMELINE

2009

Boost::format
German only



STARTUP CULTURE

STARTUP CULTURE

Features! More features!

Get a name!

STARTUP CULTURE

Features! More features!

Get a name!

Get the architecture right!

BOOST::FORMAT

Why?

- it's compatible to C's printf() family
- it's different from C's printf() family
- it supports **advanced** formatting specifiers
- it "creatively" uses operators to **separate** the formatting specification from the arguments
- it supports formatting of **user-defined** types
- it supports stream manipulators

Example:

```
#include <boost/format.hpp>

std::cout << boost::format("formatting specifier") % argument1 % argument2;
```

let's talk



TIMELINE

2009



Boost::format
German only

TIMELINE



STRING TRANSLATION

STRING TRANSLATION

Questions

STRING TRANSLATION

Questions

STRING TRANSLATION

Questions

- which translation system?

STRING TRANSLATION

Questions

- which translation system?
- is it **mature**?

STRING TRANSLATION

Questions

- which translation system?
- is it **mature**?
- does it have an ecosystem?
 - **editors**
 - tools
 - build system

STRING TRANSLATION

Questions

- which translation system?
- is it **mature**?
- does it have an ecosystem?
 - **editors**
 - tools
 - build system
- does it **fit**? Does it feel "natural"?

STRING TRANSLATION

can it be used by end users?

- domain experts
- not even remotely firm in IT

TRANSLATION FORMAT

TRANSLATION FORMAT

Binary

TRANSLATION FORMAT

~~Binary~~

TRANSLATION FORMAT

~~Binary~~

XML

TRANSLATION FORMAT

~~Binary~~

~~XML~~

TRANSLATION FORMAT

~~Binary~~

~~XML~~

JSON

TRANSLATION FORMAT

~~Binary~~

~~XML~~

~~JSON~~

TRANSLATION FORMAT

~~Binary~~

~~XML~~

~~JSON~~

Text

TRANSLATION FORMAT

~~Binary~~

~~XML~~

~~JSON~~

Text

GETTEXT

pretty standard, available on all major platforms

two major implementations

- GNU gettext
- **Boost::locale** (since Boost 1.48)

both rely on **C locales** (or their C++ equivalents) and **message catalogs**

GETTEXT

~~pretty~~ standard, available on all major platforms

two major implementations

- GNU gettext (new: POSIX 1103.1-2024, ISO/IEC 9985-1:2024 🎉)
- Boost::locale (since Boost 1.48)

both rely on C locales (or their C++ equivalents) and message catalogs

TOOLS

step 1: mark translatable text

TOOLS

step 1: **mark** translatable text

```
#include <boost/format.hpp>

std::cout << boost::format("formatting specifier") % argument1 % argument2;
```

TOOLS

step 1: **mark** translatable text

```
#include <boost/format.hpp>

std::cout << boost::format("formatting specifier") % argument1 % argument2;
```



```
#include <boost/format.hpp>
#include <boost/locale/message.hpp>
using boost::message::translate

std::cout << boost::format(translate("formatting specifier")) % argument1 % argument2;
```



TOOLS

step 2: **scan** source files

TOOLS

step 2: **scan** source files,
using 'xgettext' for all marked texts

```
$(Keyword) is e.g. "translate"
```

```
xgettext --keyword --keyword=$(Keyword):1,1t --keyword=$(Keyword):1c,2,2t  
--keyword=$(Keyword):1,2,3t --keyword=$(Keyword):1c,2,3,4t  
a.cpp b.cpp c.hpp ...
```

TOOLS

step 2: **scan** source files,
using 'xgettext' for all marked texts

```
$(Keyword) is e.g. "translate"
```

```
xgettext --keyword --keyword=$(Keyword):1,1t --keyword=$(Keyword):1c,2,2t  
--keyword=$(Keyword):1,2,3t --keyword=$(Keyword):1c,2,3,4t  
a.cpp b.cpp c.hpp ...
```

Result: a so-called PO **template** file (.pot) with all found texts

- language **agnostic**
- transient

MO' TOOLS

step 3: **translate** the strings

MO' TOOLS

step 3: **translate** the strings,
create the translation files, one for each **country, region, and company specialties**

- msguniq
- msgmerge
- msgattrib

MO' TOOLS

step 3: **translate** the strings,
create the translation files, one for each **country, region, and company specialties**

- msguniq
- msgmerge
- msgattrib

The result is a collection of **PO (portable object) files**

- specific to language, country, region, company
- **stable**
- checked into repositories

you can have **multiple** of them, e.g. one for each subsystem

PO FILE CONTENT

```
1 #
2 msgid ""
3 msgstr ""
4 "Project-Id-Version: Example 1.0\n"
5 "Report-Msgid-Bugs-To: somebody@example.com\n"
6 "POT-Creation-Date: \n"
7 "PO-Revision-Date: 2024-04-30 11:38+0200\n"
8 "Last-Translator: EMAIL@ADDRESS\n"
9 "Language-Team: Czech\n"
10 "Language: cs\n"
11 "MIME-Version: 1.0\n"
12 "Content-Type: text/plain; charset=UTF-8\n"
13 "Content-Transfer-Encoding: 8bit\n"
14 "Plural-Forms: nplurals=3; plural=(n==1 ? 0 : n>=2 && n<=4 ? 1 : 2);\n"
15 "X-Generator: Poedit 3.4.2\n"
16
17 # This is only a example
18 #: main.cpp
19 msgctxt "Disambiguation"
20 msgid "Singular"
21 msgid_plural "Plural"
22 msgstr[0] "Singularní"
23 msgstr[1] "Duální"
24 msgstr[2] "Plurál"
```

PO FILE CONTENT

```
1 #
2 msgid ""
3 msgstr ""
4 "Project-Id-Version: Example 1.0\n"
5 "Report-Msgid-Bugs-To: somebody@example.com\n"
6 "POT-Creation-Date: \n"
7 "PO-Revision-Date: 2024-04-30 11:38+0200\n"
8 "Last-Translator: EMAIL@ADDRESS\n"
9 "Language-Team: Czech\n"
10 "Language: cs\n"
11 "MIME-Version: 1.0\n"
12 "Content-Type: text/plain; charset=UTF-8\n"
13 "Content-Transfer-Encoding: 8bit\n"
14 "Plural-Forms: nplurals=3; plural=(n==1 ? 0 : n>=2 && n<=4 ? 1 : 2);\n"
15 "X-Generator: Poedit 3.4.2\n"
16
17 # This is only a example
18 #: main.cpp
19 msgctxt "Disambiguation"
20 msgid "Singular"
21 msgid_plural "Plural"
22 msgstr[0] "Singulární"
23 msgstr[1] "Duální"
24 msgstr[2] "Plurál"
```

PO FILE CONTENT

```
1 #
2 msgid ""
3 msgstr ""
4 "Project-Id-Version: Example 1.0\n"
5 "Report-Msgid-Bugs-To: somebody@example.com\n"
6 "POT-Creation-Date: \n"
7 "PO-Revision-Date: 2024-04-30 11:38+0200\n"
8 "Last-Translator: EMAIL@ADDRESS\n"
9 "Language-Team: Czech\n"
10 "Language: cs\n"
11 "MIME-Version: 1.0\n"
12 "Content-Type: text/plain; charset=UTF-8\n"
13 "Content-Transfer-Encoding: 8bit\n"
14 "Plural-Forms: nplurals=3; plural=(n==1 ? 0 : n>=2 && n<=4 ? 1 : 2);\n"
15 "X-Generator: Poedit 3.4.2\n"
16
17 # This is only a example
18 #: main.cpp
19 msgctxt "Disambiguation"
20 msgid "Singular"
21 msgid_plural "Plural"
22 msgstr[0] "Singularní"
23 msgstr[1] "Duální"
24 msgstr[2] "Plurál"
```

PO FILE CONTENT

```
1 #
2 msgid ""
3 msgstr ""
4 "Project-Id-Version: Example 1.0\n"
5 "Report-Msgid-Bugs-To: somebody@example.com\n"
6 "POT-Creation-Date: \n"
7 "PO-Revision-Date: 2024-04-30 11:38+0200\n"
8 "Last-Translator: EMAIL@ADDRESS\n"
9 "Language-Team: Czech\n"
10 "Language: cs\n"
11 "MIME-Version: 1.0\n"
12 "Content-Type: text/plain; charset=UTF-8\n"
13 "Content-Transfer-Encoding: 8bit\n"
14 "Plural-Forms: nplurals=3; plural=(n==1 ? 0 : n>=2 && n<=4 ? 1 : 2);\n"
15 "X-Generator: Poedit 3.4.2\n"
16
17 # This is only a example
18 #: main.cpp
19 msgctxt "Disambiguation"
20 msgid "Singular"
21 msgid_plural "Plural"
22 msgstr[0] "Singularní"
23 msgstr[1] "Duální"
24 msgstr[2] "Plurál"
```

EVEN MORE TOOLS

step 4: **compile** the target files

EVEN MORE TOOLS

step 4: **compile** the target files

(optionally) combine multiple PO files, and put the output into the target directory

- msgmerge
- msgfmt

'msgfmt' checks the
consistency and validity of
format specifiers and
placeholders!

MO FILE CONTENT

The result is a collection of **MO (machine object) files**

- specific to language, country, region, company
- binary
- optimized for consumption
- distributed to the customer's machine

The format is described in the manual:

<https://www.gnu.org/software/gettext/manual/gettext.html#MO-Files>

```
+-----+
byte  0 | magic number = 0x950412de
      4 | file format revision = 0
      8 | number of strings          == N
     12 | offset of table with original strings == O
     16 | offset of table with translation strings == T
     20 | size of hashing table      == S
     24 | offset of hashing table    == H
      . |
      . | (possibly more entries later)
      . |
      0 | length & offset 0th string -----
O + 8 | length & offset 1st string -----
      ...
O + ((N-1)*8) | length & offset (N-1)th string
      T | length & offset 0th translation -----
T + 8 | length & offset 1st translation -----
      ...
T + ((N-1)*8) | length & offset (N-1)th translation
      H | start hash table
      ...
H + S * 4 | end hash table
      NUL terminated 0th string <-----
      NUL terminated 1st string <-----
      ...
      NUL terminated 0th translation <-----
      NUL terminated 1st translation <-----
      ...
+-----+
```

UNICODE



About Unicode

Technical Quick Start Guide

Support Unicode +

Adopt a Character +

Membership +

News and Events +

Emoji +

Newsletter Signup ↗

 Search ...



Everyone in the world should be able to use their own language on phones and computers.

 [LEARN MORE ABOUT UNICODE](#)



[ADOPT A CHARACTER ↗](#)



TECHNICAL WORK

- > [Technical Quick Start Guide](#)
- > [Unicode Technical Site](#) ↗
- > [Public Review Issues](#) ↗
- > [Code Charts](#) ↗
- > [UTC Document Register](#) ↗
- > [The Unicode Standard](#) ↗
- > [CLDR](#) ↗
- > [ICU](#) ↗
- > [About Emoji](#)

UNICODE

Unicode covers more than glyphs, code points, and their encodings!



UNICODE

[About Unicode](#)
[Technical Quick Start Guide](#)
[Support Unicode](#) +
[Adopt a Character](#) +
[Membership](#) +
[News and Events](#) +
[Emoji](#) +
[Newsletter Signup](#) ↗

Everyone in the world should be able to use their own language on phones and computers.

[▶ LEARN MORE ABOUT UNICODE](#)

[ADOPT A CHARACTER](#) ↗

 U+4E41	 U+0644	 U+FE39	 U+3142	 U+261E	 U+1F23A	 U+1F9D9	 U+1D54
 U+30C9	 U+270C	 U+13EA	 U+0295	 U+098C	 U+056E	 U+1F639	 U+1F4A5
							
 U+261C	 U+1F643	 U+FF9F	 U+267A	 U+0E27	 U+104E	 U+CDF5	 U+02C6
 U+0C24	 U+263A	 U+FF8D	 U+01D0	 U+0D26	 U+04B3	 U+03BC	 U+05D3

TECHNICAL WORK

- > [Technical Quick Start Guide](#)
- > [Unicode Technical Site](#) ↗
- > [Public Review Issues](#) ↗
- > [Code Charts](#) ↗
- > [UTC Document Register](#) ↗
- > [The Unicode Standard](#) ↗
- > [CLDR](#) ↗
- > [ICU](#) ↗
- > [About Emoji](#)

UNICODE

Unicode covers more than glyphs, code points, and their encodings!

CLDR

Common
Language
Data
Repository

The screenshot shows the Unicode website interface. On the left is a navigation menu with links: About Unicode, Technical Quick Start Guide, Support Unicode, Adopt a Character, Membership, News and Events, Emoji, and Newsletter Signup. The main content area features the Unicode logo, a quote about global language use, and a grid of 24 character tiles, each displaying a glyph and its code point (e.g., U+4E41, U+0644, U+FE39, U+3142, U+261E, U+1F23A, U+1F9D9, U+1D54, U+30C9, U+270C, U+13EA, U+0295, U+098C, U+056E, U+1F639, U+1F4A5, U+261C, U+1F643, U+FF9F, U+267A, U+0E27, U+104E, U+CDF5, U+02C6, U+0C24, U+263A, U+FF8D, U+01D0, U+0D26, U+04B3, U+03BC, U+05D3). A search bar is located at the bottom left. A blue sidebar on the right lists technical resources under the heading 'TECHNICAL WORK', including links to the Quick Start Guide, Technical Site, Public Review Issues, Code Charts, UTC Document Register, The Unicode Standard, CLDR, ICU, and About Emoji. A yellow arrow points from the CLDR text on the left towards the 'CLDR' link in the sidebar.

CLDR

The CLDR defines machine readable rules for e.g. translation

 Unicode CLDR

Plurals & Units

Plurals

In CLDR, Plurals are used for localized Units and Compact numbers (under Numbers).

In the Survey Tool for translation purpose, plural forms shown per language will differ as only those that are relevant to that language are shown.

For example, in French, the distinction of the One and Other are available. Please see [Plural Rules](#), and [file a ticket](#) if you see a form in the Survey Tool that is not expected in your language.

Minute				
long-displayName	minutes		✓	minutes ☆
long-one	{0} minute	ⓔ	✓	{0} minute ☆
long-other	{0} minutes	ⓔ	✓	{0} minutes ☆
long-per	{0} per minute		✓	{0} par minute ☆
short-displayName	mins		✓	min ☆
short-one	{0} min	ⓔ	✓	{0} min ☆
short-other	{0} min	ⓔ	✓	{0} min ☆
short-per	{0}/min		✓	{0}/min ☆
narrow-displayName	min		✓	min ☆

CLDR

The CLDR defines machine readable rules for e.g. translation

→ Plurals & Units

 Unicode CLDR

Plurals

In CLDR, Plurals are used for localized Units and Compact numbers (under Numbers).

In the Survey Tool for translation purpose, plural forms shown per language will differ as only those that are relevant to that language are shown.

For example, in French, the distinction of the One and Other are available. Please see [Plural Rules](#), and [file a ticket](#) if you see a form in the Survey Tool that is not expected in your language.

Minute				
long-displayName	minutes		✓	minutes ☆
long-one	{0} minute	ⓔ	✓	{0} minute ☆
long-other	{0} minutes	ⓔ	✓	{0} minutes ☆
long-per	{0} per minute		✓	{0} par minute ☆
short-displayName	mins		✓	min ☆
short-one	{0} min	ⓔ	✓	{0} min ☆
short-other	{0} min	ⓔ	✓	{0} min ☆
short-per	{0}/min		✓	{0}/min ☆
narrow-displayName	min		✓	min ☆

CLDR

The CLDR defines machine readable rules for e.g. translation

  Unicode CLDR



Plurals & Units

Plurals

In CLDR, Plurals are used for localized Units and Compact numbers (under Numbers).

In the Survey Tool for translation purpose, plural forms shown per language will differ as only those that are relevant to that language are shown.

For example, in French, the distinction of the One and Other are available. Please see [Plural Rules](#) 

Tool that is not expected in your language.

Minute				
long-displayName	minutes		✓	minutes ☆
long-one	{0} minute	ⓔ	✓	{0} minute ☆
long-other	{0} minutes	ⓔ	✓	{0} minutes ☆
long-per	{0} per minute		✓	{0} par minute ☆
short-displayName	mins		✓	min ☆
short-one	{0} min	ⓔ	✓	{0} min ☆
short-other	{0} min	ⓔ	✓	{0} min ☆
short-per	{0}/min		✓	{0}/min ☆
narrow-displayName	min		✓	min ☆

LANGUAGE PLURAL RULES

Languages vary in plural forms

UNCLDR Charts

Home | Site Map | Search

CLDR v45.0

Language Plural Rules

2024-04-16

[Index](#)

Languages vary in how they handle plurals of nouns or unit expressions ("hours", "meters", and so on). Some languages have two forms, like English; some languages have only a single form; and some languages have multiple forms (see [Slovenian](#) below). They also vary between cardinals (such as 1, 2, or 3) and ordinals (such as 1st, 2nd, or 3rd), and in ranges of cardinals (such as "1-2", used in expressions like "1-2 meters long"). CLDR uses short, mnemonic tags for these plural categories. For more information on these categories, see [Plural Rules](#).

- Examples:** The symbol ~ (as in "1.7~2.1") has a special meaning: it is a range of numbers that includes the end points (1.7 and 2.1), and everything between that has exactly the same number of decimals as the end points (thus also 1.8, 1.9, and 2.0, but not 2 or 1.91 or 1.90). The samples are generated mechanically, and are not comprehensive: "0, 2~19, 101~119, ..." could show up as the less-complete "0, 2~16, 101 ...".
- Rules:** The plural categories are computed based on machine-readable rules, using the syntax described in [Language Plural Rules](#). In particular, they use special variables and relation defined in [Plural Rule Operands](#) and following.
- Reporting Defects:** When you find errors or omissions in this data, please report the information with a [bug report](#). But first read "Reporting Defects" on [Plural Rules](#).

Contents

1. [Rules](#)

2. [Comparison](#)

1. Rules

Name	Code	Type	Category	Examples	Minimal Pairs	Rules
Afrikaans	af	cardinal	one	1 1.0, 1.00, 1.000, 1.0000	1 dag 1,0 dag	n = 1
			other	0, 2~16, 100, 1000, 10000, 100000, 1000000, ... 0.0~0.9, 1.1~1.6, 10.0, 100.0, 1000.0, 10000.0, 100000.0, 1000000.0, ...	2 dae 0,9 dae	
		ordinal	other	0~15, 100, 1000, 10000, 100000, 1000000, ...	Neem die 15e afdraai na regs.	
		range	n/a	n/a	Not available. Please file a ticket to supply.	n/a

LANGUAGE PLURAL RULES

Languages vary in plural forms



UN

CLDR Charts

[Home](#) | [Site Map](#) | [Search](#)

CLDR v45.0

Language Plural Rules

2024-04-16

[Index](#)

Languages vary in how they handle plurals of nouns or unit expressions ("hours", "meters", and so on). Some languages have two forms, like English; some languages have only a single form; and some languages have multiple forms (see [Slovenian](#) below). They also vary between cardinals (such as 1, 2, or 3) and ordinals (such as 1st, 2nd, or 3rd), and in ranges of cardinals (such as "1-2", used in expressions like "1-2 meters long"). CLDR uses short, mnemonic tags for these plural categories. For more information on these categories, see [Plural Rules](#).

- **Examples:** The symbol ~ (as in "1.7~2.1") has a special meaning: it is a range of numbers that includes the end points (1.7 and 2.1), and everything between that has exactly the same number of decimals as the end points (thus also 1.8, 1.9, and 2.0, but not 2 or 1.91 or 1.90). The samples are generated mechanically, and are not comprehensive: "0, 2~19, 101~119, ..." could show up as the less-complete "0, 2~16, 101 ...".
- **Rules:** The plural categories are computed based on machine-readable rules, using the syntax described in [Language Plural Rules](#). In particular, they use special variables and relation defined in [Plural Rule Operands](#) and following.
- **Reporting Defects:** When you find errors or omissions in this data, please report the information with a [bug report](#). But first read "Reporting Defects" on [Plural Rules](#).

Contents

1. [Rules](#)

2. [Comparison](#)

1. Rules

Name	Code	Type	Category	Examples	Minimal Pairs	Rules
Afrikaans	af	cardinal	one	1 1.0, 1.00, 1.000, 1.0000	1 dag 1,0 dag	n = 1
			other	0, 2~16, 100, 1000, 10000, 100000, 1000000, ... 0.0~0.9, 1.1~1.6, 10.0, 100.0, 1000.0, 10000.0, 100000.0, 1000000.0, ...	2 dae 0,9 dae	
		ordinal	other	0~15, 100, 1000, 10000, 100000, 1000000, ...	Neem die 15e afdraai na regs.	
		range	n/a	n/a	Not available. Please file a ticket to supply.	n/a

LANGUAGE PLURAL RULES

Languages vary in plural forms



ISO 639
ISO 3166,
POSIX



UNCLDR Charts

Home | Site Map | Search

CLDR v45.0

Language Plural Rules

2024-04-16

[Index](#)

Languages vary in how they handle plurals of nouns or unit expressions ("hours", "meters", and so on). Some languages have two forms, like English; some languages have only a single form; and some languages have multiple forms (see [Slovenian](#) below). They also vary between cardinals (such as 1, 2, or 3) and ordinals (such as 1st, 2nd, or 3rd), and in ranges of cardinals (such as "1-2", used in expressions like "1-2 meters long"). CLDR uses short, mnemonic tags for these plural categories. For more information on these categories, see [Plural Rules](#).

- Examples:** The symbol ~ (as in "1.7~2.1") has a special meaning: it is a range of numbers that includes the end points (1.7 and 2.1), and everything between that has exactly the same number of decimals as the end points (thus also 1.8, 1.9, and 2.0, but not 2 or 1.91 or 1.90). The samples are generated mechanically, and are not comprehensive: "0, 2~19, 101~119, ..." could show up as the less-complete "0, 2~16, 101 ...".
- Rules:** The plural categories are computed based on machine-readable rules, using the syntax described in [Language Plural Rules](#). In particular, they use special variables and relation defined in [Plural Rule Operands](#) and following.
- Reporting Defects:** When you find errors or omissions in this data, please report the information with a [bug report](#). But first read "Reporting Defects" on [Plural Rules](#).

Contents

1. [Rules](#)

2. [Comparison](#)

1. Rules

Name	Code	Type	Category	Examples	Minimal Pairs	Rules
Afrikaans	af	cardinal	one	1 1.0, 1.00, 1.000, 1.0000	1 dag 1,0 dag	n = 1
			other	0, 2~16, 100, 1000, 10000, 100000, 1000000, ... 0.0~0.9, 1.1~1.6, 10.0, 100.0, 1000.0, 10000.0, 100000.0, 1000000.0, ...	2 dae 0,9 dae	
		ordinal	other	0~15, 100, 1000, 10000, 100000, 1000000, ...	Neem die 15e afdraai na regs.	
		range	n/a	n/a	Not available. Please file a ticket to supply.	n/a

LANGUAGE PLURAL RULES

Languages vary in plural forms



ISO 639
ISO 3166,
POSIX



UNCLDR Charts

Home | Site Map | Search

CLDR v45.0

Language Plural Rules

2024-04-16

[Index](#)

Languages vary in how they handle plurals of nouns or unit expressions ("hours", "meters", and so on). Some languages have two forms, like English; some languages have only a single form; and some languages have multiple forms (see [Slovenian](#) below). They also vary between cardinals (such as 1, 2, or 3) and ordinals (such as 1st, 2nd, or 3rd), and in ranges of cardinals (such as "1-2", used in expressions like "1-2 meters long"). CLDR uses short, mnemonic tags for these plural categories. For more information on these categories, see [Plural Rules](#).

- **Examples:** The symbol ~ (as in "1.7~2.1") has a special meaning: it is a range of numbers that includes the end points (1.7 and 2.1), and everything between that has exactly the same number of decimals as the end points (thus also 1.8, 1.9, and 2.0, but not 2 or 1.91 or 1.90). The samples are generated mechanically, and are not comprehensive: "0, 2~19, 101~119, ..." could show up as the less-complete "0, 2~16, 101 ...".
- **Rules:** The plural categories are computed based on machine-readable rules, using the syntax described in [Language Plural Rules](#). In particular, they use special variables and relation defined in [Plural Rule Operands](#) and following.
- **Reporting Defects:** When you find errors or omissions in this data, please report the information with a [bug report](#). But first read "Reporting Defects" on [Plural Rules](#).

Contents

1. [Rules](#)

2. [Comparison](#)

1. Rules

Name	Code	Type	Category	Examples	Minimal Pairs	Rules
Afrikaans	af	cardinal	one	1 1.0, 1.00, 1.000, 1.0000	1 dag 1,0 dag	n = 1
			other	0, 2~16, 100, 1000, 10000, 100000, 1000000, ... 0.0~0.9, 1.1~1.6, 10.0, 100.0, 1000.0, 10000.0, 100000.0, 1000000.0, ...	2 dae 0,9 dae	
		ordinal	other	0~15, 100, 1000, 10000, 100000, 1000000, ...	Neem die 15e afdraai na regs.	
		range	n/a	n/a	Not available. Please file a ticket to supply.	n/a

LANGUAGE PLURAL RULES

Languages vary in plural forms

ISO 639
ISO 3166,
POSIX

UNCLDR Charts

Home | Site Map | Search

CLDR v45.0

Language Plural Rules

2024-04-16

[Index](#)

Languages vary in how they handle plurals of nouns or unit expressions ("hours", "meters", and so on). Some languages have two forms, like English; some languages have only a single form; and some languages have multiple forms (see [Slovenian](#) below). They also vary between cardinals (such as 1, 2, or 3) and ordinals (such as 1st, 2nd, or 3rd), and in ranges of cardinals (such as "1-2", used in expressions like "1-2 meters long"). CLDR uses short, mnemonic tags for these plural categories. For more information on these categories, see [Plural Rules](#).

- **Examples:** The symbol ~ (as in "1.7~2.1") has a special meaning: it is a range of numbers that includes the end points (1.7 and 2.1), and everything between that has exactly the same number of decimals as the end points (thus also 1.8, 1.9, and 2.0, but not 2 or 1.91 or 1.90). The samples are generated mechanically, and are not comprehensive: "0, 2~19, 101~119, ..." could show up as the less-complete "0, 2~16, 101 ...".
- **Rules:** The plural categories are computed based on machine-readable rules, using the syntax described in [Language Plural Rules](#). In particular, they use special variables and relation defined in [Plural Rule Operands](#) and following.
- **Reporting Defects:** When you find errors or omissions in this data, please report the information with a [bug report](#). But first read "Reporting Defects" on [Plural Rules](#).

Contents

1. [Rules](#)

2. [Comparison](#)

1. Rules

Name	Code	Type	Category	Examples	Minimal Pairs	Rules
Afrikaans	af	cardinal	one	1 1.0, 1.00, 1.000, 1.0000	1 dag 1,0 dag	n = 1
			other	0, 2~16, 100, 1000, 10000, 100000, 1000000, ... 0.0~0.9, 1.1~1.6, 10.0, 100.0, 1000.0, 10000.0, 100000.0, 1000000.0, ...	2 dae 0,9 dae	
		ordinal	other	0~15, 100, 1000, 10000, 100000, 1000000, ...	Neem die 15e afdraai na regs.	
		range	n/a	n/a	Not available. Please file a ticket to supply.	n/a

from none, to simple, to complicated

0	1	2	3	4	5	6	7-9	9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100									
1 Bambara, Burmese, Cantonese, Chinese, Dzongkha, Hmong Njua, Igbo, Indonesian, Japanese, Javanese, Kabuverdianu, Khmer, Korean, Koyraboro Senni, Lakota, Lao, Lojban, Makonde, Malay, N'Ko, Osage, Root, Sango, Sichuan Yi, Sundanese, Tagalog, Tatar, Thai, Vietnamese, Wolof, Xhosa, Yoruba, Zulu																																																																																																			
0	1	2	3	4	5	6	7-9	9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100									
2 Cebuano, Filipino, Tagalog																																																																																																			
0	1	2	3	4	5	6	7-9	9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100									
2 Central Atlas Tamazight																																																																																																			
0	1	2	3	4	5	6	7-9	9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100									
2 Icelandic, Macedonian																																																																																																			
0	1	2	3	4	5	6	7-9	9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100									
2 Akan, Amharic, Armenian, Assamese, Bangla, Bhojpuri, Dogri, French, Fula, Gujarati, Gun, Hindi, Kabyle, Kannada, Lingala, Malagasy, Nigerian Pidgin, Northern Sotho, Persian, Portuguese, Punjabi, Sinhala, Tigrinya, Walloon, Zulu																																																																																																			
0	1	2	3	4	5	6	7-9	9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70																																				

Figure 1 displays a grid of 30 rows, each representing a language. The first column lists the language names. The subsequent 59 columns show a sequence of 'x' and 'o' characters, with some characters highlighted in blue or green. The languages are: Czech, Slovak, Manx, Scottish Gaelic, Breton, Lower Sorbian, Slovenian, Upper Sorbian, Maltese, and Irish. The 'x' and 'o' characters represent different phonetic or morphological features across the languages.

from none, to simple, to complicated

[illegible]

from none, to simple, to complicated

0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
1 Bambara, Burmese, Cantonese, Chinese, Dzongkha, Hmong Njua, Igbo, Indonesian, Japanese, Javanese, Kabuverdianu, Khmer, Korean, Koyraboro Senni, Lakota, Lao, Lojban, Makonde, Malay, N'Ko, Osage, Root, Sango, Sichuan Yi, Sundanese, Tagalog, Tswana, Vietnamese, Wolof, Xhosa, Yoruba, Zulu																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Cebuano, Filipino, Tagalog																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Central Atlas Tamazight																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Icelandic, Macedonian																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Akan, Amharic, Armenian, Assamese, Bangla, Bhojpuri, Dogri, French, Fula, Gujarati, Gun, Hindi, Kabyle, Kannada, Lingala, Malagasy, Nigerian Pidgin, Northern Sotho, Persian, Portuguese, Punjabi, Sinhala, Tigrinya, Walloon, Zulu																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76																																

[illegible]

from none, to simple, to complicated

0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
1 Bambara, Burmese, Cantonese, Chinese, Dzongkha, Hmong Njua, Igbo, Indonesian, Japanese, Javanese, Kabuverdianu, Khmer, Korean, Koyraboro Senni, Lakota, Lao, Lojban, Makonde, Malay, N'Ko, Osage, Root, Sango, Sichuan Yi, Sundanese, Tagalog, Tswana, Vietnamese, Wolof, Xhosa, Yiddish, Zulu																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Cebuano, Filipino, Tagalog																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Central Atlas Tamazight																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Icelandic, Macedonian																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	76	77-78	79	80	81	82	83	84	85	86	87-88	89	90	91	92	93	94	95	96	97-98	99	100											
2 Akan, Amharic, Armenian, Assamese, Bangla, Bhojpuri, Dogri, French, Fula, Gujarati, Gun, Hindi, Kabyle, Kannada, Lingala, Malagasy, Nigerian Pidgin, Northern Sotho, Persian, Portuguese, Punjabi, Sinhala, Tigrinya, Walloon, Zulu																																																																																																				
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-18	19	20	21	22	23	24	25	26	27-28	29	30	31	32	33	34	35	36	37-38	39	40	41	42	43	44	45	46	47-48	49	50	51	52	53	54	55	56	57-58	59	60	61	62	63	64	65	66	67-68	69	70	71	72	73	74	75	7																																

[illegible]

from none, to simple, to complicated

[illegible]

Figure 1 displays the distribution of 120 languages across 120 time steps (0-119). The languages are listed on the left, and their presence is indicated by colored blocks in the grid. The languages are: Czech, Slovak; Manx; Scottish Gaelic; Breton; Lower Sorbian, Slovenian, Upper Sorbian; Maltese; and Irish. The colors represent different categories: orange, green, blue, purple, and yellow. The grid shows that most languages are present for a significant duration, with some appearing later in the timeline.

PLURAL FORMS

GNU **gettext** tries to cover these patterns in **single-line C** expressions.

Most implementations follow the CLDR rules.

```
1 "Language: ja"
2 "Plural-Forms: nplurals=1; plural=0;"
3
4 "Language: de"
5 "Plural-Forms: nplurals=2; plural=(n != 1);"
6
7 "Language: cs"
8 "Plural-Forms: nplurals=3; plural=(n==1 ? 0 : n>=2 && n<=4 ? 1 : 2);"
9
10 "Language: br"
11 "Plural-Forms: nplurals=5; plural=(n%10==1 && n%100!=11 && n%100!=71 && n%100!=91 ? 0 : n%10==2 && n%100!=12 &&
n%100!=72 && n%100!=92 ? 1 : ((n%10>=3 && n%10<=4) || n%10==9) && (n%100<10 || n%100>19) && (n%100<70 || n%100>79)
&& (n%100<90 || n%100>99) ? 2 : n!=0 && n%1000000==0 ? 3 : 4);"
```

[illegible]

PLURAL FORMS

GNU **gettext** tries to cover these patterns in **single-line C** expressions.

Most implementations follow the CLDR rules.

```

1 "Language: ja"
2 "Plural-Forms: nplurals=1; plural=0;"
3
4 "Language: de"
5 "Plural-Forms: nplurals=2; plural=(n != 1);"
6
7 "Language: cs"
8 "Plural-Forms: nplurals=3; plural=(n==1 ? 0 : n>=2 && n<=4 ? 1 : 2);"
9
10 "Language: br"
11 "Plural-Forms: nplurals=5; plural=(n%10==1 && n%100!=11 && n%100!=71 && n%100!=91 ? 0 : n%10==2 && n%100!=12 &&
n%100!=72 && n%100!=92 ? 1 : ((n%10>=3 && n%10<=4) || n%10==9) && (n%100<10 || n%100>19) && (n%100<70 || n%100>79)
&& (n%100<90 || n%100>99) ? 2 : n!=0 && n%1000000==0 ? 3 : 4);"

```

3	Czech, Slovak	x o f x																																																																						
0	1	2	3	4	5	6	7-9	10	11	12	13	14	15	16	17-19	20	21	22	23	24	25	26	27-29	30	31	32	33	34	35	36	37-39	40	41	42	43	44	45	46	47-49	50	51	52	53	54	55	56	57-59	60	61	62	63	64	65	66	67-69	70	71	72	73	74	75	76	77-79	80	81	82	83	84	85	86	87-89	90
4	Manx	f o x																																																																						

BOOST.LOCALE

BOOST.LOCALE

it does too much!

BOOST.LOCALE

it does too much!

on **every** return from a translation call,

- it does a **translation**

BOOST.LOCALE

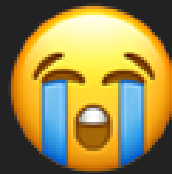
it does too much!

on **every** return from a translation call,

- it does a **translation**
- it does an **encoding conversion**

BOOST.LOCALE

it does too much!



on **every** return from a translation call,

- it does a **translation**
- it does an **encoding conversion**

therefore it **requires a string allocation**

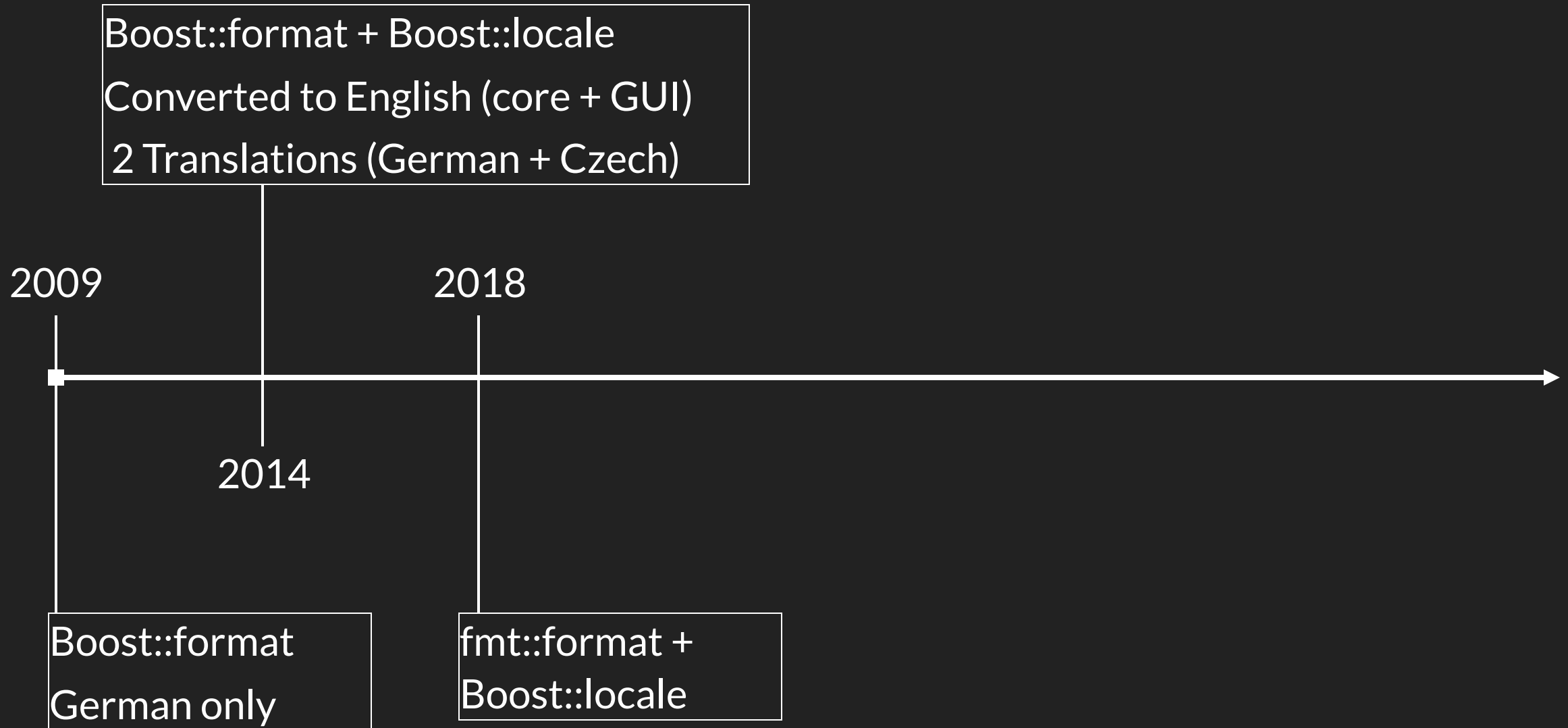
a new library



TIMELINE



TIMELINE



{FMT}

```
#include <boost/format.hpp>
```

```
std::cout << boost::format("formatting specifier") % argument1 % argument2;
```

```
1 template <typename Char>  
2 format(const something-related-to-Char & FS) -> boost::basic_format<Char>;  
3     +  
4 template <typename Char, typename T>  
5 operator%(boost::basic_format<Char> &, const T &) -> boost::basic_format<Char>;
```

{FMT}

```
#include <boost/format.hpp>
```

```
std::cout << boost::format("formatting specifier") % argument1 % argument2;
```



```
#include <fmt/format.hpp>
```

```
std::cout << fmt::format("formatting specifier", argument1, argument2);
```

```
1 template <typename Char>  
2 format(const something-related-to-Char & FS) -> boost::basic_format<Char>;  
3     +  
4 template <typename Char, typename T>  
5 operator%(boost::basic_format<Char> &, const T &) -> boost::basic_format<Char>;
```



```
1 template <typename... Ts>  
2 format(std::string_view FS, Ts &&... Args) -> std::string;  
3  
4 template <typename... Ts>  
5 format(std::wstring_view FS, Ts &&... Args) -> std::wstring;
```

{FMT} HAS TWO INTERFACES!

```
1 // Interface 1: typeful
2
3 template <typename... Types>
4 format(std::string_view, Types &&... Args) -> std::string;
5
6
7 // Interface 2: type-erased
8
9 vformat(std::string_view, format_args) -> std::string;
10
11
12 // the typeful interface is a thin wrapper around the type-erased interface
13
14 template <typename... Types>
15 format(std::string_view FS, Types &&... Args -> std::string {
16     return vformat(FS, make_format_args(Args));
17 }
```

{FMT} HAS TWO INTERFACES!

```
1 // Interface 1: typeful
2
3 template <typename... Types>
4 format(std::string_view, Types &&... Args) -> std::string;
5
6
7 // Interface 2: type-erased
8
9 vformat(std::string_view, format_args) -> std::string;
10
11
12 // the typeful interface is a thin wrapper around the type-erased interface
13
14 template <typename... Types>
15 format(std::string_view FS, Types &&... Args -> std::string {
16     return vformat(FS, make_format_args(Args));
17 }
```

{FMT} HAS TWO INTERFACES!

```
1 // Interface 1: typeful
2
3 template <typename... Types>
4 format(std::string_view, Types &&... Args) -> std::string;
5
6
7 // Interface 2: type-erased
8
9 vformat(std::string_view, format_args) -> std::string;
10
11
12 // the typeful interface is a thin wrapper around the type-erased interface
13
14 template <typename... Types>
15 format(std::string_view FS, Types &&... Args -> std::string {
16     return vformat(FS, make_format_args(Args));
17 }
```

{FMT} HAS TWO INTERFACES!

```
1 // Interface 1: typeful
2
3 template <typename... Types>
4 format(std::string_view, Types &&... Args) -> std::string;
5
6
7 // Interface 2: type-erased
8
9 vformat(std::string_view, format_args) -> std::string;
10
11
12 // the typeful interface is a thin wrapper around the type-erased interface
13
14 template <typename... Types>
15 format(std::string_view FS, Types &&... Args -> std::string {
16     return vformat(FS, make_format_args(Args));
17 }
```



Type erasure, or better: **type classification**

TYPE ERASURE

{fmt} is using all kinds of **type-based** metaprogramming to achieve its goals:

TYPE ERASURE

{fmt} is using all kinds of **type-based** metaprogramming to achieve its goals:

- **templates** (type calculations)

TYPE ERASURE

{fmt} is using all kinds of **type-based** metaprogramming to achieve its goals:

- **templates** (type calculations)
- **partial template specializations** (i.e. "pattern matching")

TYPE ERASURE

{fmt} is using all kinds of **type-based** metaprogramming to achieve its goals:

- **templates** (type calculations)
- **partial template specializations** (i.e. "pattern matching")
- **overload** resolution and **conversion** sequences of unimplemented functions (conversion-based metaprogramming)

TYPE ERASURE

{fmt} is using all kinds of **type-based** metaprogramming to achieve its goals:

- **templates** (type calculations)
- **partial template specializations** (i.e. "pattern matching")
- **overload** resolution and **conversion** sequences of unimplemented functions (conversion-based metaprogramming)
- **SFINAE** ("substitution failure is not an error") to control overload sets and viability of functions

RULE OF CHIEL

COST OF OPERATIONS

- SFINAE
- Instantiating a function template
- Instantiating a type
- Calling an alias
- Adding a parameter to a type
- Adding a parameter to an alias call
- looking up a memoized type

AKA THE RULE OF CHIEL

RULE OF Chiel Douwes 💜

COST OF OPERATIONS

- SFINAE
- Instantiating a function template
- Instantiating a type
- Calling an alias
- Adding a parameter to a type
- Adding a parameter to an alias call
- looking up a memoized type

AKA THE RULE OF CHIEL

ONE MORE THING...

ONE MORE THING...

Boost.MP11

ONE MORE THING...

Boost.MP11

Kvasir

ONE MORE THING...

Boost.MP11

use libraries!

Kvasir

ONE MORE THING...

Boost.MP11

use libraries!

Kvasir

Brigand

ONE MORE THING...

Boost.MP11

Kvasir

Meta

use libraries!

Brigand

ONE MORE THING...

Boost.MP11

~~Boost.MPL~~

Meta

use libraries!

Kvasir

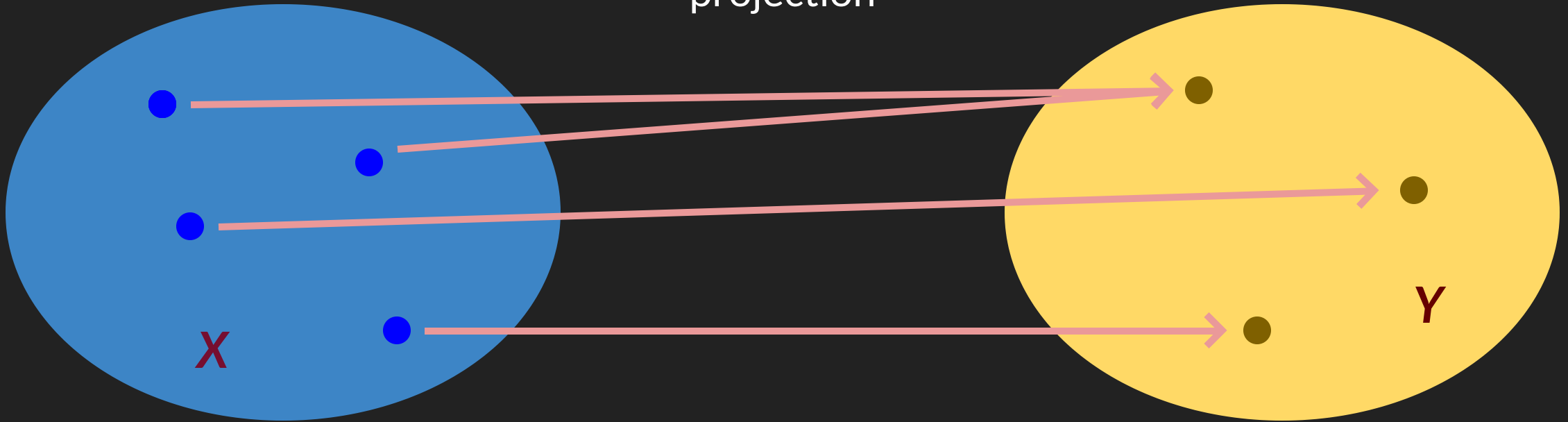
Brigand

**WHAT IS A
FUNCTION?**

$$y = f(x)$$

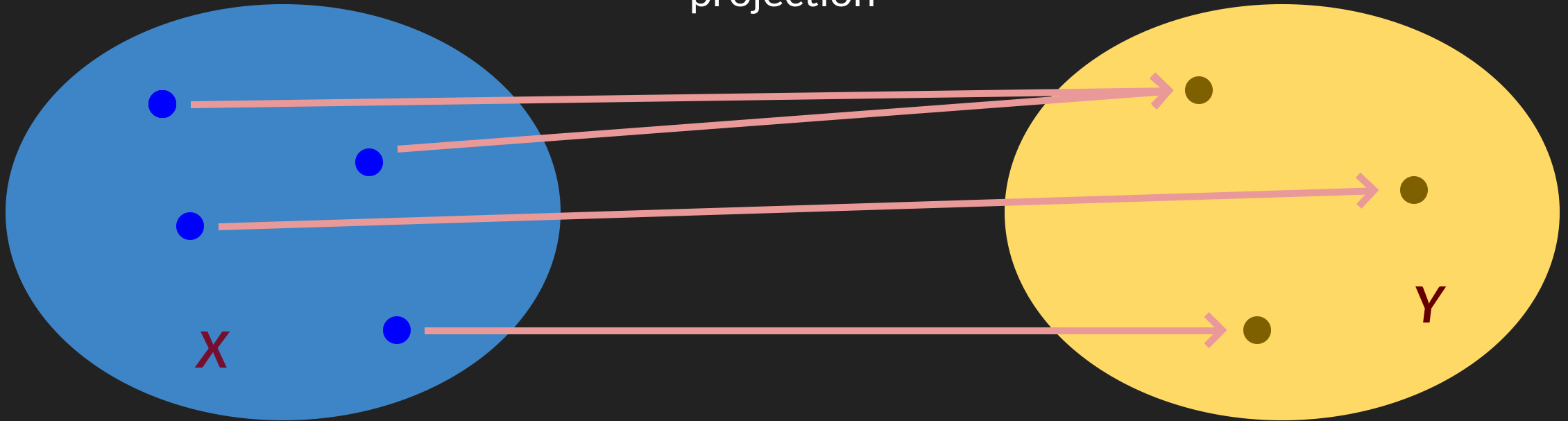
WHAT IS A FUNCTION?

the operator f describes **any** kind of projection



WHAT IS A FUNCTION?

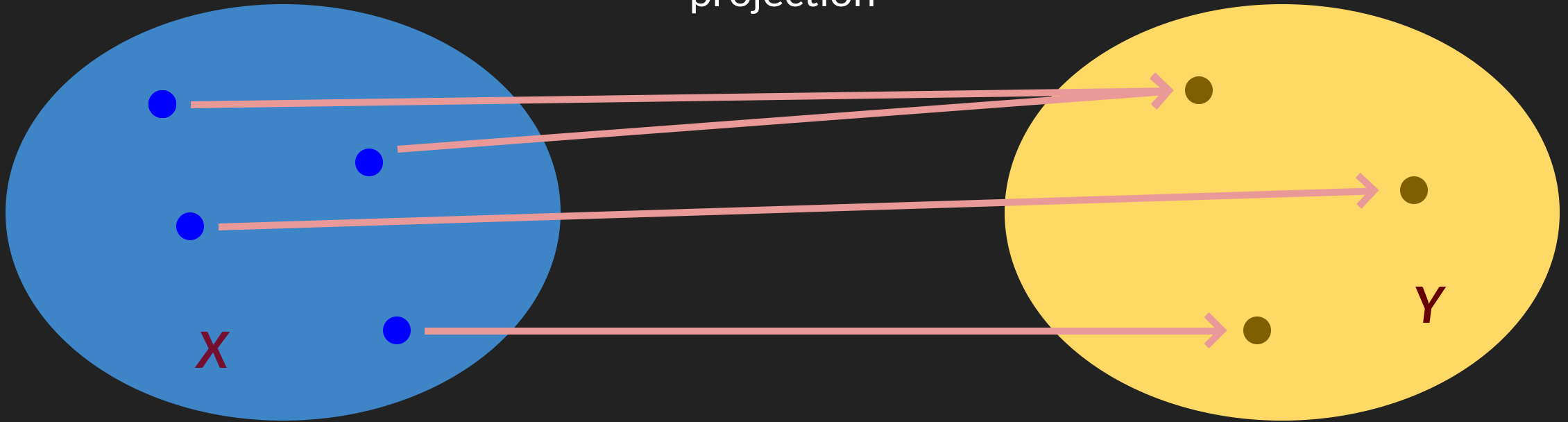
the operator f describes **any** kind of projection



example $f: x \rightarrow \begin{cases} y_0 & \text{if } x \text{ matches predicate 0} \\ y_1 & \text{if } x \text{ matches predicate 1} \\ \dots & \text{if } x \text{ matches predicate } \dots \end{cases}$

WHAT IS A FUNCTION?

the operator f describes **any** kind of projection



anything,

not necessarily just
one element

anything else,

necessarily exactly
one element

example $f: x \rightarrow \begin{cases} y_0 & \text{if } x \text{ matches predicate 0} \\ y_1 & \text{if } x \text{ matches predicate 1} \\ \dots & \text{if } x \text{ matches predicate } \dots \end{cases}$

WHAT IS A FUNCTION?

$f()$



WHAT IS A FUNCTION?

$f()$



$g()$

WHAT IS A FUNCTION?

$f()$



$g()$



WHAT IS A FUNCTION?

$f()$



$g()$



$\hbar()$

WHAT IS A FUNCTION?

$f()$



$g()$



$h()$



WHAT IS A FUNCTION?

$f()$



$g()$



$h()$



$\text{aunt_Anji}\nexists()$

WHAT IS A FUNCTION?

$f()$



$g()$



$\hbar()$



$a_{\text{unt_Aññi}} \#()$



WHAT IS A FUNCTION?

$f()$ ✓

$g()$ ✓

$\hbar()$ ✓

$a_{\ddot{u}nt_A\ddot{n}\ddot{h}i\ddot{z}}()$ ✓

  $()$

some unutterable name

WHAT IS A FUNCTION?

$f()$



$g()$



$\hbar()$



$a_{\ddot{u}nt_A\ddot{n}\ddot{h}i\ddot{z}}()$



 $()$



some unutterable name

WHAT IS A FUNCTION?

names are **unimportant** as long as they are **unique**

$f()$



$g()$



$\hbar()$



$a_{\ddot{u}nt_A\ddot{n}\ddot{h}i\ddot{z}}()$



 $()$



some unutterable name

WHAT IS A FUNCTION?

$$y = f(x)$$

WHAT IS A FUNCTION?

$$y = f(x)$$



$$x \longrightarrow y$$

WHAT IS A FUNCTION?

$$y = f(x)$$



$$x \longrightarrow y$$

$$y = f\{x\}$$

WHAT IS A FUNCTION?

$$y = f(x)$$



$$x \longrightarrow y$$

$$y = f\{x\}$$



$$x \longrightarrow y$$

WHAT IS A FUNCTION?

$$y = f(x)$$



$$x \longrightarrow y$$

$$y = f\{x\}$$



$$x \longrightarrow y$$

$$y = f[x]$$

WHAT IS A FUNCTION?

$y = f(x)$		$x \longrightarrow y$
------------	---	-----------------------

$y = f\{x\}$		$x \longrightarrow y$
--------------	---	-----------------------

$y = f[x]$		$x \longrightarrow y$
------------	---	-----------------------

WHAT IS A FUNCTION?

$y = f(x)$		$x \longrightarrow y$
------------	---	-----------------------

$y = f\{x\}$		$x \longrightarrow y$
--------------	---	-----------------------

$y = f[x]$		$x \longrightarrow y$
------------	---	-----------------------

$$y = f\llbracket x \rrbracket$$

WHAT IS A FUNCTION?

$y = f(x)$	✓	$x \longrightarrow y$
$y = f\{x\}$	✓	$x \longrightarrow y$
$y = f[x]$	✓	$x \longrightarrow y$
$y = f\llbracket x \rrbracket$	✓	$x \longrightarrow y$

WHAT IS A FUNCTION?

$y = f(x)$	✓	$x \longrightarrow y$
$y = f\{x\}$	✓	$x \longrightarrow y$
$y = f[x]$	✓	$x \longrightarrow y$
$y = f\langle x \rangle$	✓	$x \longrightarrow y$
$y = f<x>$		

WHAT IS A FUNCTION?

$y = f(x)$	✓	$x \longrightarrow y$
$y = f\{x\}$	✓	$x \longrightarrow y$
$y = f[x]$	✓	$x \longrightarrow y$
$y = f\langle x \rangle$	✓	$x \longrightarrow y$
$y = f<x>$	✓	$x \longrightarrow y$

WHAT IS A FUNCTION?

$$y = f(x) \quad \checkmark \quad x \longrightarrow y$$

$$y = f\{x\} \quad \checkmark \quad x \longrightarrow y$$

$$y = f[x] \quad \checkmark \quad x \longrightarrow y$$

$$y = f\langle x \rangle \quad \checkmark \quad x \longrightarrow y$$

$$y = f\langle x \rangle \quad \checkmark \quad x \longrightarrow y$$

$$y = f\langle x_0 \rangle(x_1)$$

WHAT IS A FUNCTION?

$y = f(x)$	✓	$x \longrightarrow y$
$y = f\{x\}$	✓	$x \longrightarrow y$
$y = f[x]$	✓	$x \longrightarrow y$
$y = f\langle x \rangle$	✓	$x \longrightarrow y$
$y = f\langle x \rangle$	✓	$x \longrightarrow y$
$y = f\langle x_0 \rangle(x_1)$	✓	$x_0, x_1 \longrightarrow y$

WHAT IS A FUNCTION?

spellings are **unimportant**, but they may have **context**

$$y = f(x) \quad \checkmark \quad x \longrightarrow y$$

$$y = f\{x\} \quad \checkmark \quad x \longrightarrow y$$

$$y = f[x] \quad \checkmark \quad x \longrightarrow y$$

$$y = f\langle x \rangle \quad \checkmark \quad x \longrightarrow y$$

$$y = f\langle x \rangle \quad \checkmark \quad x \longrightarrow y$$

$$y = f\langle x_0 \rangle(x_1) \quad \checkmark \quad x_0, x_1 \longrightarrow y$$

WHAT IS A FUNCTION?

$$y = f(x)$$

$$y = f_{<x_0>}(x_1)$$

$$y = f_{<x>}()$$

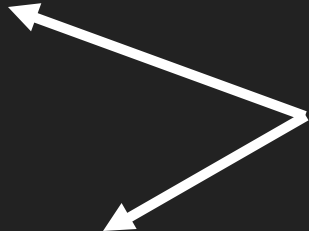
WHAT IS A FUNCTION?

$$y = f(x)$$

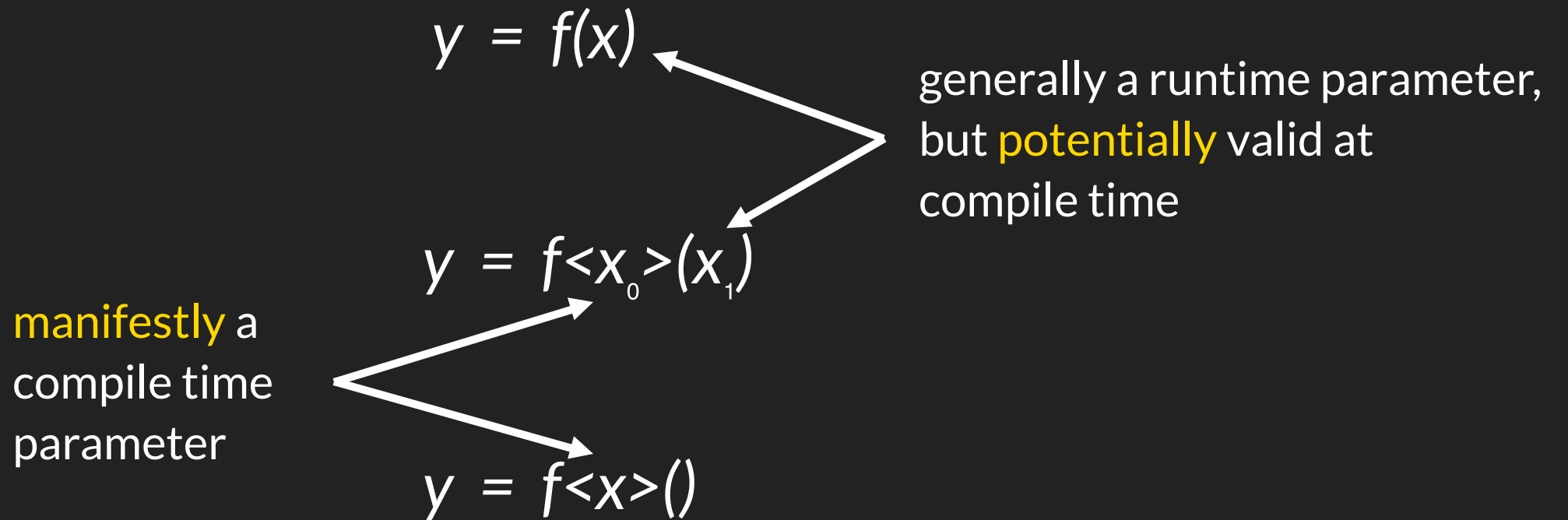
$$y = f\langle x_0 \rangle(x_1)$$

$$y = f\langle x \rangle()$$

generally a runtime parameter,
but **potentially** valid at
compile time

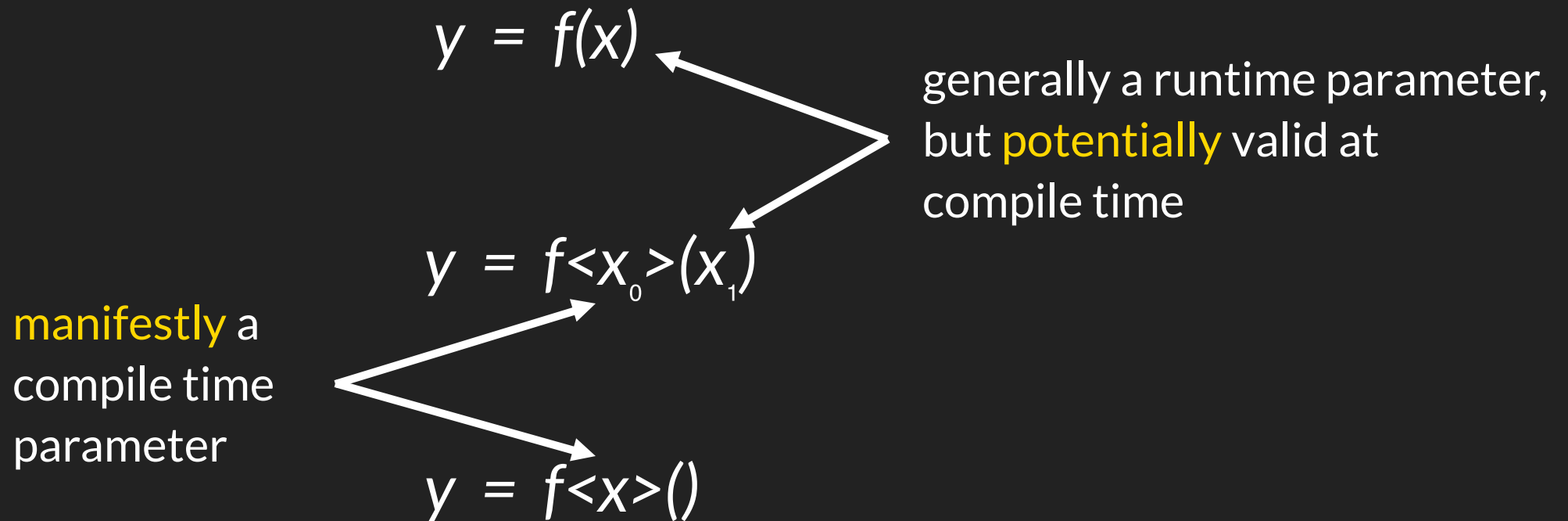


WHAT IS A FUNCTION?



WHAT IS A FUNCTION?

the **context** determines in which parameters are **valid**



WHAT IS A FUNCTION?

std::is_object<T>

WHAT IS A FUNCTION?

std::is_object<*T*>  $T \longrightarrow \text{bool}$

WHAT IS A FUNCTION?

std::is_object<*T*>  $T \longrightarrow \text{bool}$

std::is_convertible<*From*, *To*>

WHAT IS A FUNCTION?

std::is_object<*T*>  $T \longrightarrow \text{bool}$

std::is_convertible<*From*, *To*>  $T_0, T_1 \longrightarrow \text{bool}$

WHAT IS A FUNCTION?

std::is_object<*T*> ✓ *T* $\longrightarrow$ *bool*

std::is_convertible<*From*, *To*> ✓ *T*₀, *T*₁ $\longrightarrow$ *bool*

std::remove_cvref<*T*>

WHAT IS A FUNCTION?

std::is_object<*T*>  $T \longrightarrow \text{bool}$

std::is_convertible<*From*, *To*>  $T_0, T_1 \longrightarrow \text{bool}$

std::remove_cvref<*T*>  $T \longrightarrow T'$

WHAT IS A FUNCTION?

std::is_object<*T*> ✓ *T* $\longrightarrow$ *bool*

std::is_convertible<*From*, *To*> ✓ *T*₀, *T*₁ $\longrightarrow$ *bool*

std::remove_cvref<*T*> ✓ *T* $\longrightarrow$ *T*'

std::common_type<*T*₀, *T*₁>

WHAT IS A FUNCTION?

std::is_object<*T*> ✓ *T* $\longrightarrow$ *bool*

std::is_convertible<*From*, *To*> ✓ *T*₀, *T*₁ $\longrightarrow$ *bool*

std::remove_cvref<*T*> ✓ *T* $\longrightarrow$ *T*'

std::common_type<*T*₀, *T*₁> ✓ *T*₀, *T*₁ $\longrightarrow$ *U*

WHAT IS A FUNCTION?

std::is_object<*T*> ✓ $T \longrightarrow \text{bool}$

std::is_convertible<*From*, *To*> ✓ $T_0, T_1 \longrightarrow \text{bool}$

std::remove_cvref<*T*> ✓ $T \longrightarrow T'$

std::common_type<*T*₀, *T*₁> ✓ $T_0, T_1 \longrightarrow U$

std::invoke_result<*f*<*Ts*...>>

WHAT IS A FUNCTION?

<i>std::is_object</i> < <i>T</i> >	✓	<i>T</i>	→	<i>bool</i>
<i>std::is_convertible</i> < <i>From</i> , <i>To</i> >	✓	<i>T</i> ₀ , <i>T</i> ₁	→	<i>bool</i>
<i>std::remove_cvref</i> < <i>T</i> >	✓	<i>T</i>	→	<i>T</i> '
<i>std::common_type</i> < <i>T</i> ₀ , <i>T</i> ₁ >	✓	<i>T</i> ₀ , <i>T</i> ₁	→	<i>U</i>
<i>std::invoke_result</i> < <i>f</i> < <i>Ts</i> ...>>	✓	<i>f</i> , <i>Ts</i> ...	→	<i>U</i>

WHAT IS A FUNCTION?

<code>std::is_object<T></code>	✓	$T \longrightarrow \text{bool}$
<code>std::is_convertible<From, To></code>	✓	$T_0, T_1 \longrightarrow \text{bool}$
<code>std::remove_cvref<T></code>	✓	$T \longrightarrow T'$
<code>std::common_type<T₀, T₁></code>	✓	$T_0, T_1 \longrightarrow U$
<code>std::invoke_result<f<Ts...>></code>	✓	$f, Ts... \longrightarrow U$
<code>std::void_t<Ts...></code>		

WHAT IS A FUNCTION?

<code>std::is_object<T></code>	✓	$T \longrightarrow \text{bool}$
<code>std::is_convertible<From, To></code>	✓	$T_0, T_1 \longrightarrow \text{bool}$
<code>std::remove_cvref<T></code>	✓	$T \longrightarrow T'$
<code>std::common_type<T₀, T₁></code>	✓	$T_0, T_1 \longrightarrow U$
<code>std::invoke_result<f<Ts...>></code>	✓	$f, Ts... \longrightarrow U$
<code>std::void_t<Ts...></code>	✓	$Ts... \longrightarrow \text{void}$

WHAT IS A FUNCTION?

the **type traits** from the standard library are your friend

<code>std::is_object<T></code>	✓	$T \longrightarrow \text{bool}$
<code>std::is_convertible<From, To></code>	✓	$T_0, T_1 \longrightarrow \text{bool}$
<code>std::remove_cvref<T></code>	✓	$T \longrightarrow T'$
<code>std::common_type<T₀, T₁></code>	✓	$T_0, T_1 \longrightarrow U$
<code>std::invoke_result<f<Ts...>></code>	✓	$f, Ts... \longrightarrow U$
<code>std::void_t<Ts...></code>	✓	$Ts... \longrightarrow \text{void}$

WHAT IS A FUNCTION?

the **type traits** from the standard library are your friend

<code>std::is_object<T></code>	✓	$T \longrightarrow \text{bool}$
<code>std::is_convertible<From, To></code>	✓	$T_0, T_1 \longrightarrow \text{bool}$
<code>std::remove_cvref<T></code>	✓	$T \longrightarrow T'$
<code>std::common_type<T₀, T₁></code>	✓	$T_0, T_1 \longrightarrow U$
<code>std::invoke_result<f<Ts...>></code>	✓	$f, Ts... \longrightarrow U$
<code>std::void_t<Ts...></code>	✓	$Ts... \longrightarrow \text{void}$
<code>std::vector<T></code>		

WHAT IS A FUNCTION?

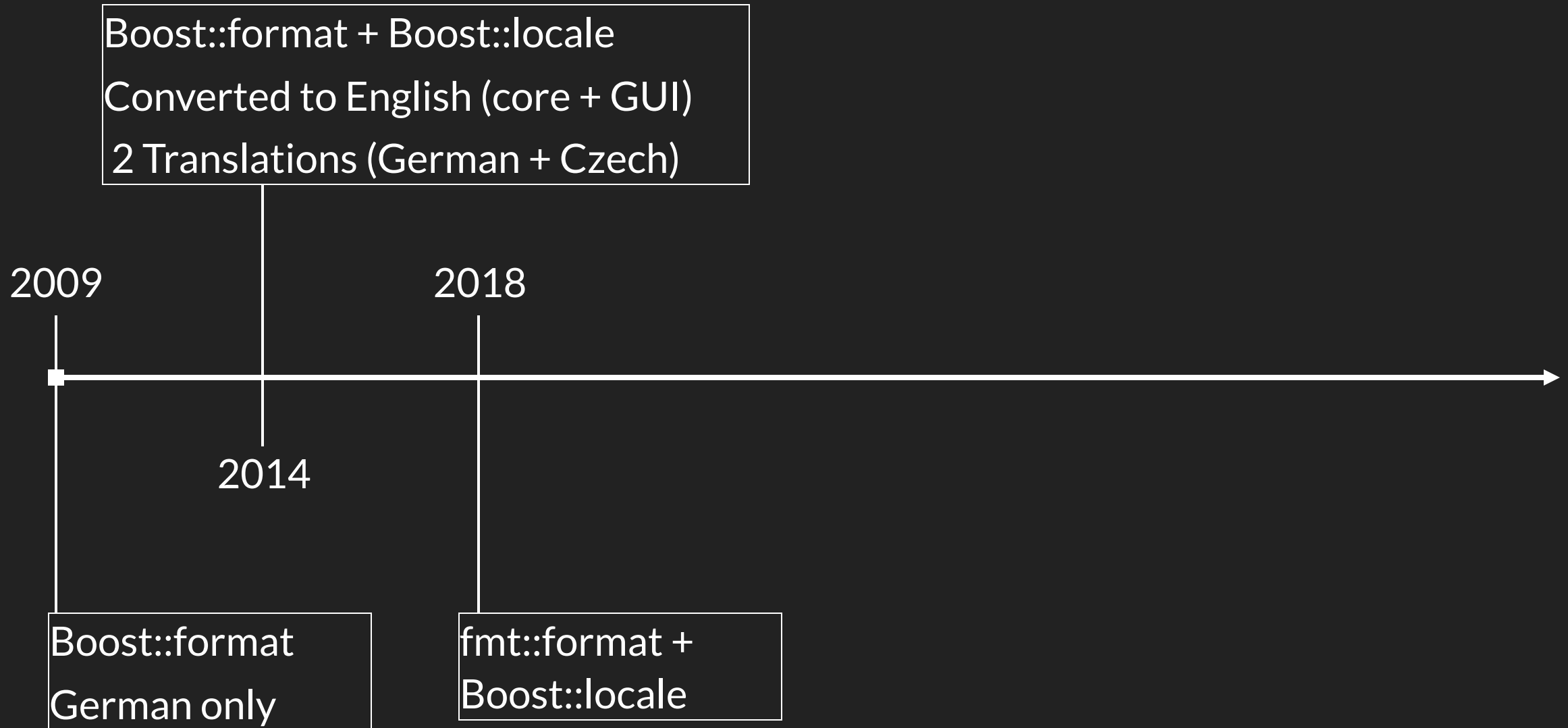
the **type traits** from the standard library are your friend

<code>std::is_object<T></code>	✓	$T \longrightarrow \text{bool}$
<code>std::is_convertible<From, To></code>	✓	$T_0, T_1 \longrightarrow \text{bool}$
<code>std::remove_cvref<T></code>	✓	$T \longrightarrow T'$
<code>std::common_type<T₀, T₁></code>	✓	$T_0, T_1 \longrightarrow U$
<code>std::invoke_result<f<Ts...>></code>	✓	$f, Ts... \longrightarrow U$
<code>std::void_t<Ts...></code>	✓	$Ts... \longrightarrow \text{void}$
<code>std::vector<T></code>	✓	$T \longrightarrow U$

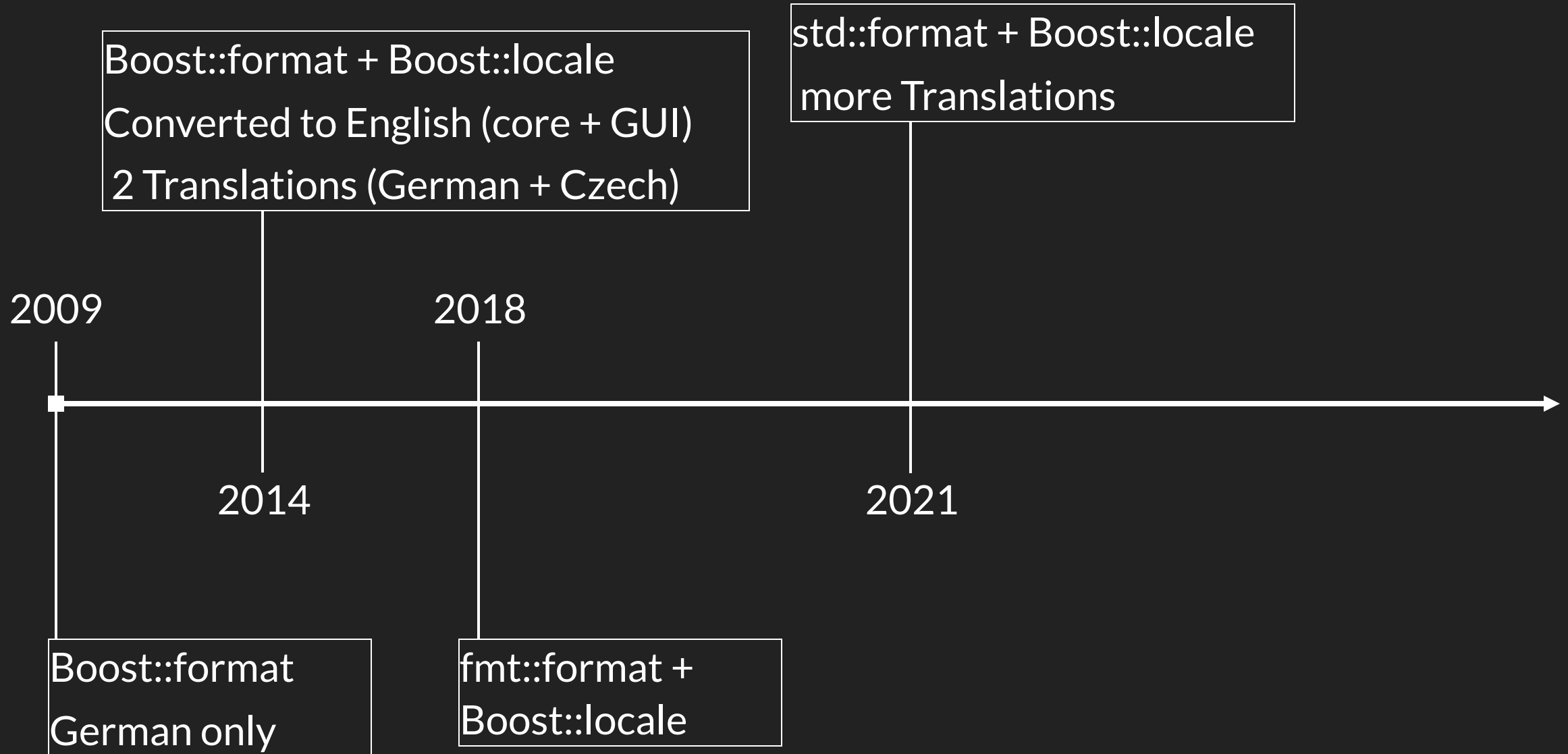
a new world



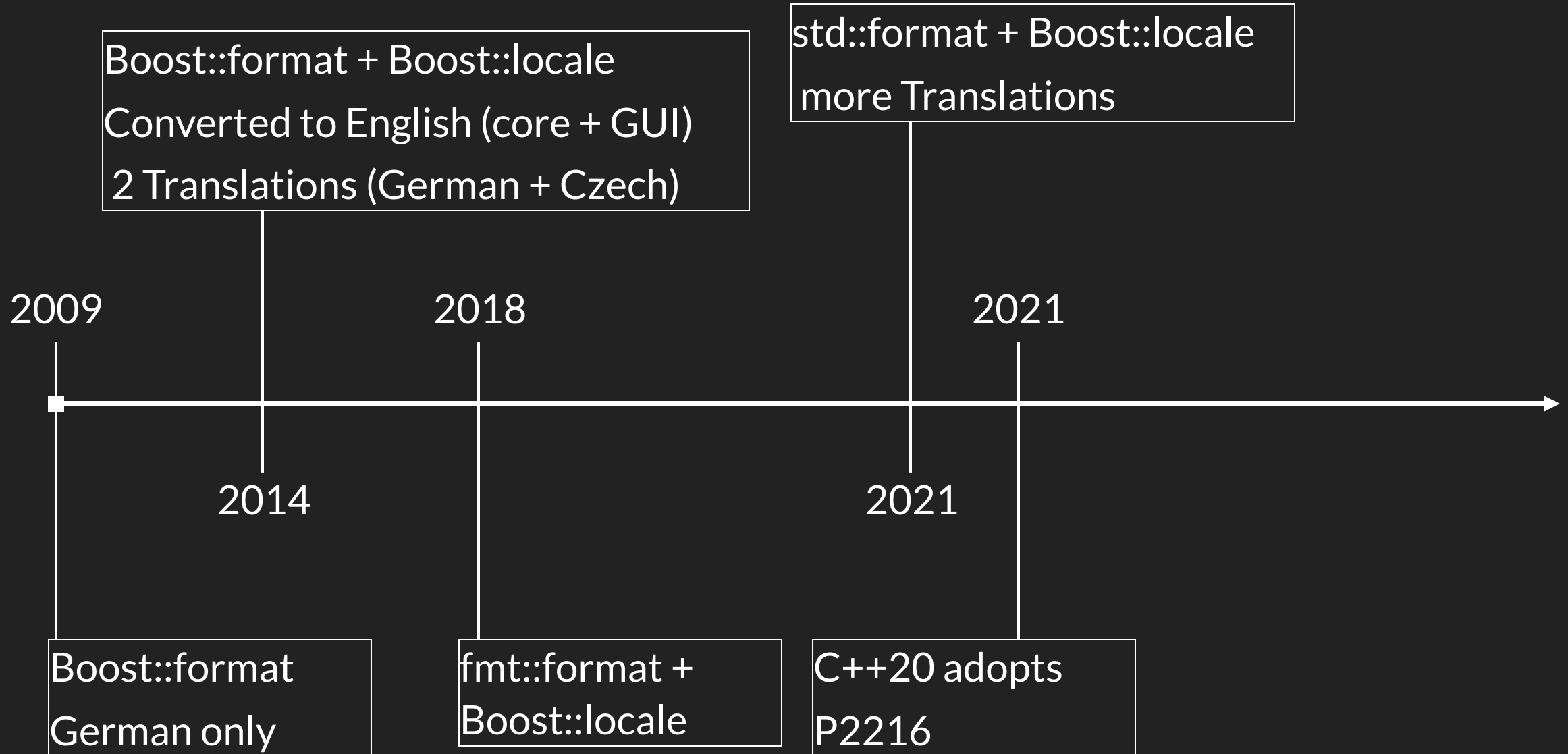
TIMELINE



TIMELINE



TIMELINE



FROM C++20

Original C++20 (Prague 2020, N4860)

```
1 template<typename... Types>
2 auto std::format(std::string_view Fmt, const Types &... Args) -> std::string {
3     return std::vformat(Fmt, std::make_format_args(Args...));
4 }
```

FROM C++20 TO C++20

Original C++20 (Prague 2020, N4860)

```
1 template<typename... Types>
2 auto std::format(std::string_view Fmt, const Types &... Args) -> std::string {
3     return std::vformat(Fmt, std::make_format_args(Args...));
4 }
```

Contemporary C++20 (April 2024, N4981)

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Types &&... Args) -> std::string {
3     return std::vformat(Fmt.get(), std::make_format_args(Args...));
4 }
5
6 template <typename... Types>
7 struct format_string {
8     constexpr format_string(std::string_view Str) : Str_(Str) {}
9
10    constexpr std::string_view get() const noexcept { return Str_; }
11
12 private:
13     std::string_view Str_;
14 };
```

FROM C++20 TO C++20

Original C++20 (Prague 2020, N4860)

```
1 template<typename... Types>
2 auto std::format(std::string_view Fmt, const Types &... Args) -> std::string {
3     return std::vformat(Fmt, std::make_format_args(Args...));
4 }
```

Contemporary C++20 (April 2024, N4981)

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Types &&... Args) -> std::string {
3     return std::vformat(Fmt.get(), std::make_format_args(Args...));
4 }
5
6 template <typename... Types>
7 struct format_string {
8     constexpr format_string(std::string_view Str) : Str_(Str) {}
9
10    constexpr std::string_view get() const noexcept { return Str_; }
11
12 private:
13     std::string_view Str_;
14 };
```

FROM C++20 TO C++20

Original C++20 (Prague 2020, N4860)

```
1 template<typename... Types>
2 auto std::format(std::string_view Fmt, const Types &... Args) -> std::string {
3     return std::vformat(Fmt, std::make_format_args(Args...));
4 }
```

Contemporary C++20 (April 2024, N4981)

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Types &&... Args) -> std::string {
3     return std::vformat(Fmt.get(), std::make_format_args(Args...));
4 }
5
6 template <typename... Types>
7 struct format_string {
8     constexpr format_string(std::string_view Str) : Str_(Str) {}
9
10    constexpr std::string_view get() const noexcept { return Str_; }
11
12 private:
13    std::string_view Str_;
14 };
```

FROM C++20 TO C++20

Original C++20 (Prague 2020, N4860)

```
1 template<typename... Types>
2 auto std::format(std::string_view Fmt, const Types &... Args) -> std::string {
3     return std::vformat(Fmt, std::make_format_args(Args...));
4 }
```

Contemporary C++20 (April 2024, N4981)

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Types &&... Args) -> std::string {
3     return std::vformat(Fmt.get(), std::make_format_args(Args...));
4 }
5
6 template <typename... Types>
7 struct format_string {
8     constexpr format_string(std::string_view Str) : Str_(Str) {}
9
10    constexpr std::string_view get() const noexcept { return Str_; }
11
12 private:
13     std::string_view Str_;
14 };
```

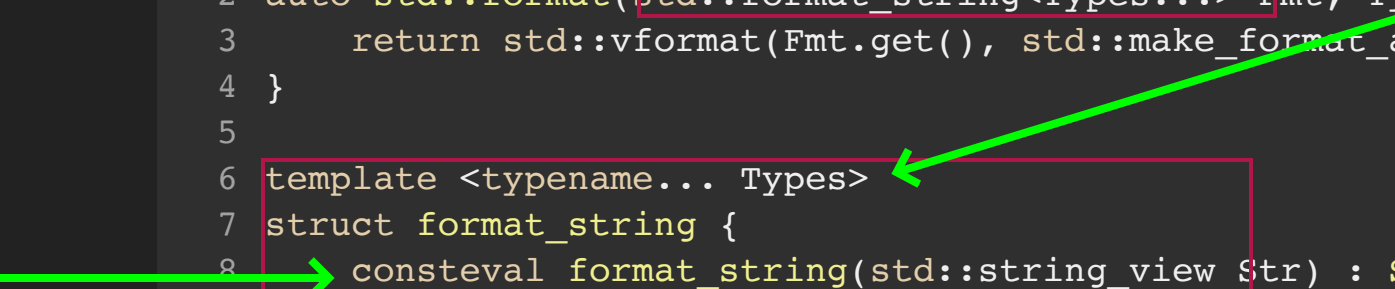
FROM C++20 TO C++20

Original C++20 (Prague 2020, N4860)

```
1 template<typename... Types>
2 auto std::format(std::string_view Fmt, const Types &... Args) -> std::string {
3     return std::vformat(Fmt, std::make_format_args(Args...));
4 }
```

Contemporary C++20 (April 2024, N4981)

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Types &&... Args) -> std::string {
3     return std::vformat(Fmt.get(), std::make_format_args(Args...));
4 }
5
6 template <typename... Types>
7 struct format_string {
8     constexpr format_string(std::string_view Str) : Str_(Str) {}
9
10     constexpr std::string_view get() const noexcept { return Str_; }
11
12 private:
13     std::string_view Str_;
14 };
```



CONSTEVAL



WHAT IS CONSTANT EVALUATION?

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities
- declarations

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities
- declarations
- definitions

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities
- declarations
- definitions
- ...

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities
- declarations
- definitions
- ...
- templates

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

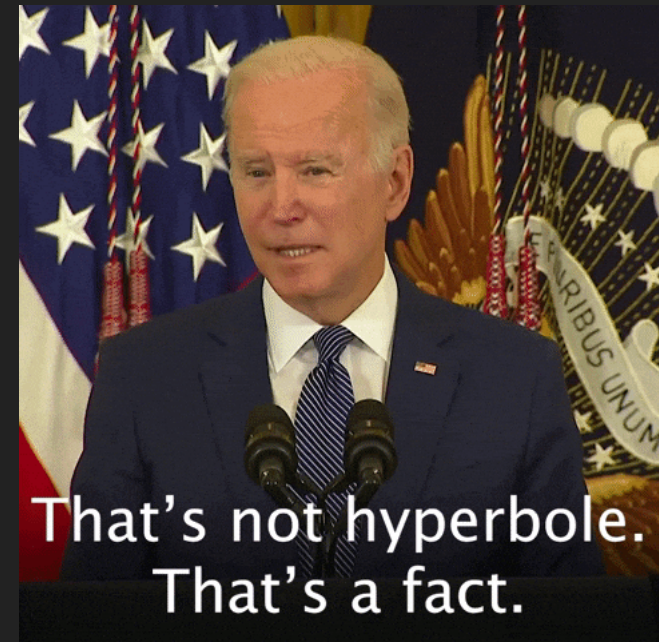
- identifiers
- entities
- declarations
- definitions
- ...
- templates
- all template instantiations so far

WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities
- declarations
- definitions
- ...
- templates
- all template instantiations so far

Pretty much everything.



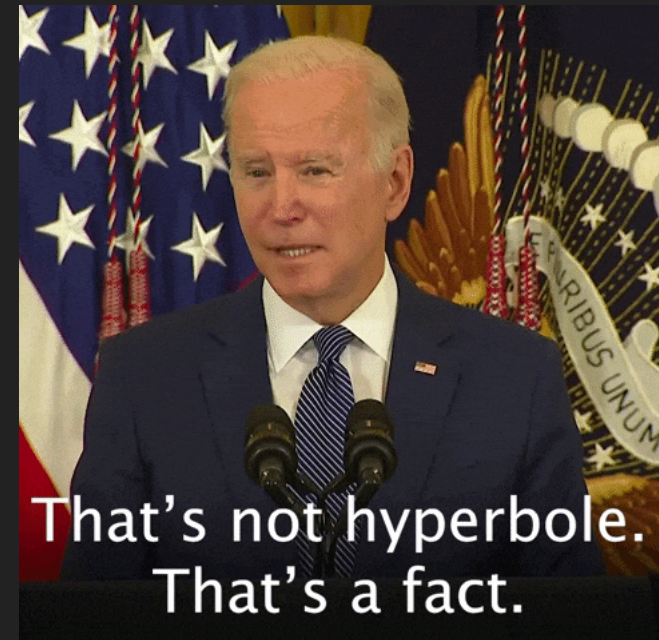
WHAT IS CONSTANT EVALUATION?

During compilation, the compiler has to remember everything it has seen so far:

- identifiers
- entities
- declarations
- definitions
- ...
- templates
- all template instantiations so far

Pretty much everything.

BTW, this is why we have modules now 🎉



WHAT IS CONSTANT EVALUATION?

What about these code lines?

WHAT IS CONSTANT EVALUATION?

What about these code lines?

- `int i = 1 + 2 + 3 / 4;`
- `bool b{ 42 == L"42" };`
- `char a[] = "huh?";`
- `enum e{ none, anyone, couple };`
- `static_assert( none != anyone);`
- `const int c = couple;`
- `std::vector<int> v = { none * 2, anyone * sizeof(a), couple * c };`
- ...

WHAT IS CONSTANT EVALUATION?

What about these code lines?

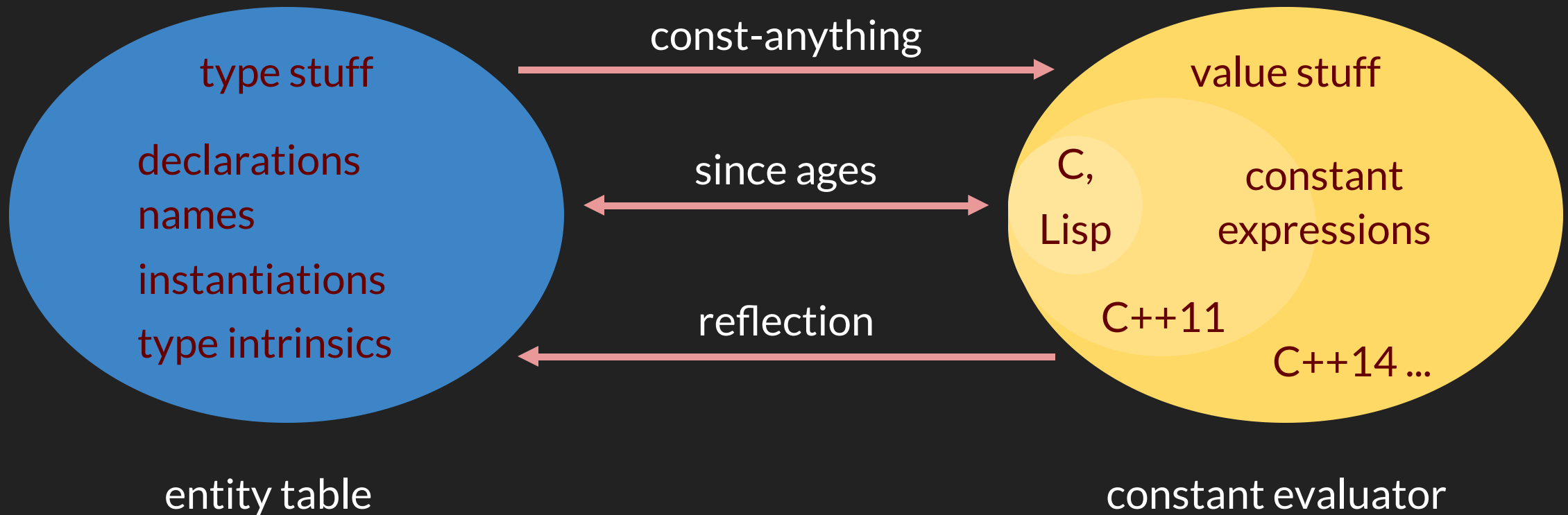
- `int i = 1 + 2 + 3 / 4;`
- `bool b{ 42 == L"42" };`
- `char a[] = "huh?";`
- `enum e{ none, anyone, couple };`
- `static_assert( none != anyone );`
- `const int c = couple;`
- `std::vector<int> v = { none * 2, anyone * sizeof(a), couple * c };`
- ...

constant required,
"constant expression"



SPLIT BRAIN

two subsystems for compile-time entities



CONSTANT EXPRESSION

Wherever a "constant" is required

a **new** constant evaluation is started



even **within** an **ongoing**
constant evaluation!



P2216

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Args &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string {
6     consteval format_string(std::string_view Str) {
7         constexpr size_t num_args = sizeof...(Types);
8         constexpr basic_format_arg_type arg_types[num_args > 0 ? num_args : 1] = {
9             std::get_format_arg_type<Types>()...
10         };
11
12         parse_format_string(Str, format_checker<std::remove_cvref_t<Types>...>{Str, arg_types});
13     }
14 };
```

P2216

```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Args &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string {
6     consteval format_string(std::string_view Str) {
7         constexpr size_t num_args = sizeof...(Types);
8         constexpr basic_format_arg_type arg_types[num_args > 0 ? num_args : 1] = {
9             std::get_format_arg_type<Types>()...
10        };
11
12        parse_format_string(Str, format_checker<std::remove_cvref_t<Types>...>{Str, arg_types});
13    }
14};
```

taken from the MS-STL implementation

P2216

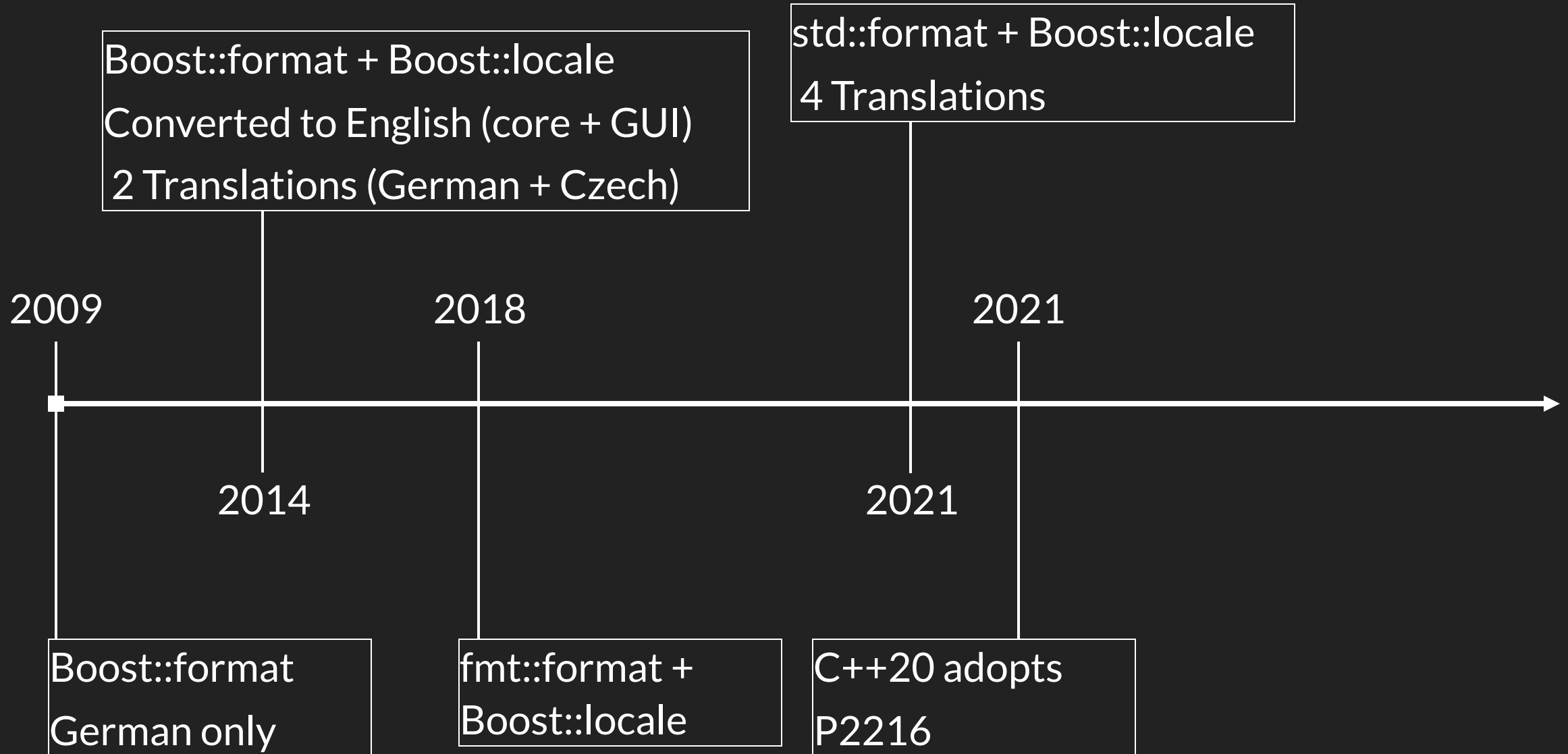
```
1 template<typename... Types>
2 auto std::format(std::format_string<Types...> Fmt, Args &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string {
6     consteval format_string(std::string_view Str) {
7         constexpr size_t num_args = sizeof...(Types);
8         constexpr basic_format_arg_type arg_types[num_args > 0 ? num_args : 1] = {
9             std::get_format_arg_type<Types>()...
10        };
11
12        parse_format_string(Str, format_checker<std::remove_cvref_t<Types>...>{Str, arg_types});
13    }
14};
```

format string **syntax** and argument **type** checking can - and will ! - be done
at compile time
no more exceptions at runtime!

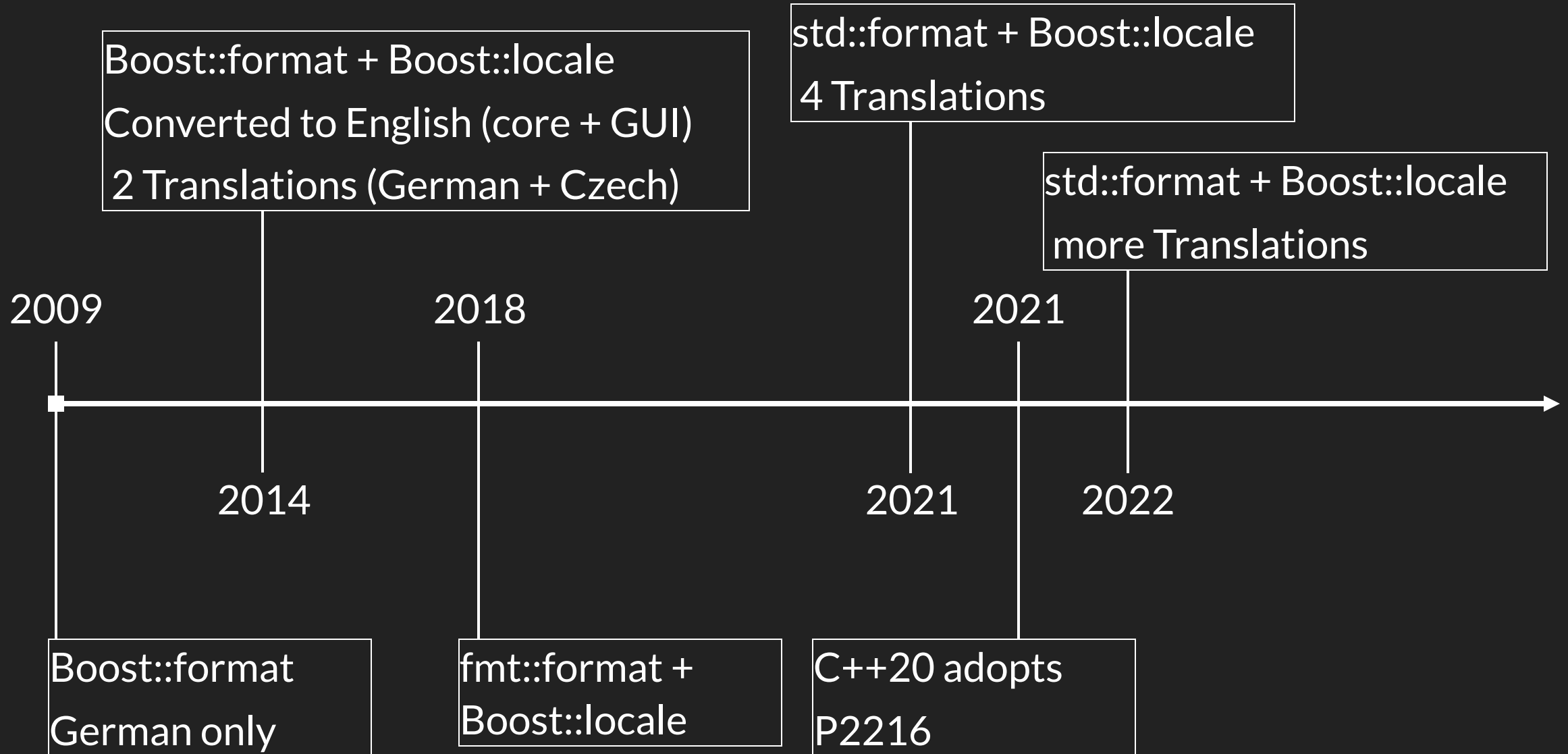
what now?



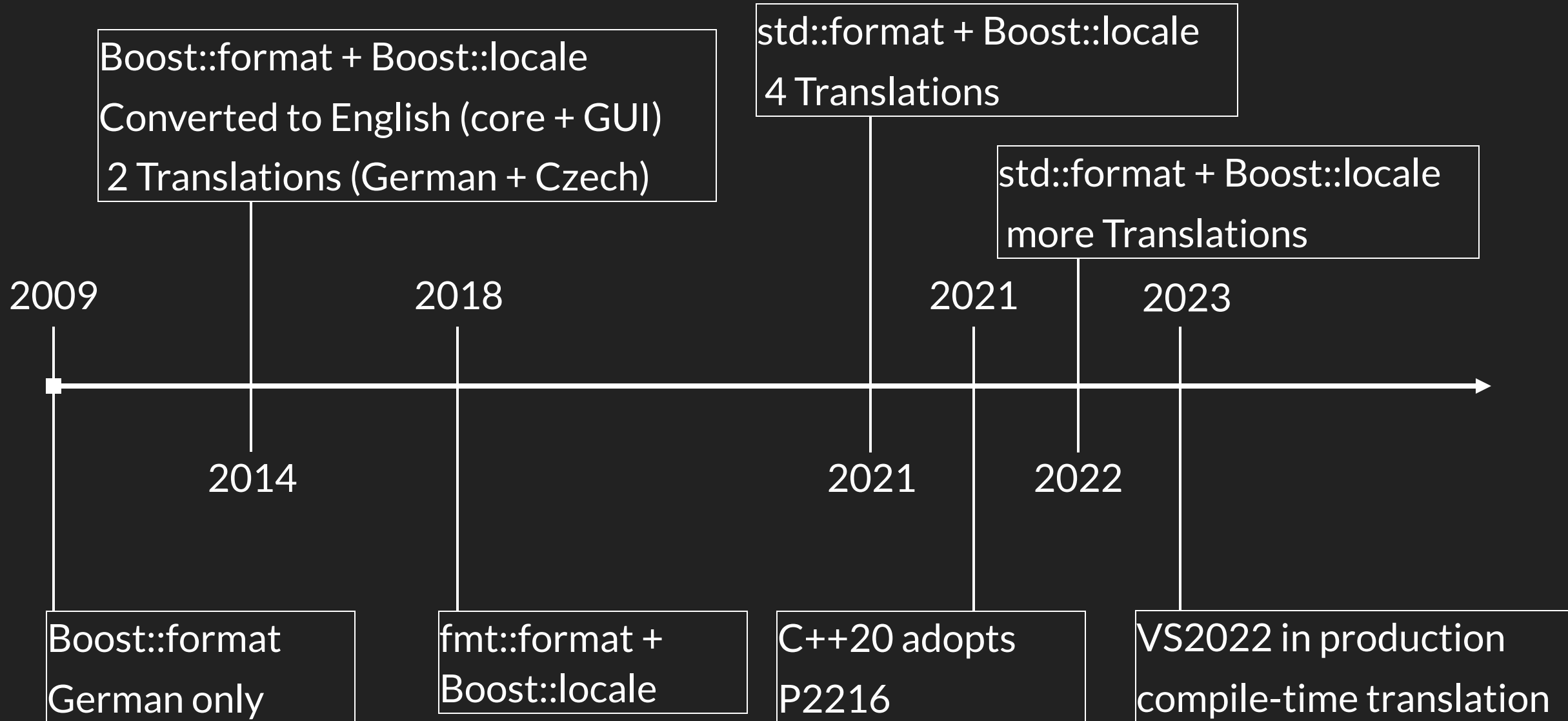
TIMELINE



TIMELINE



TIMELINE



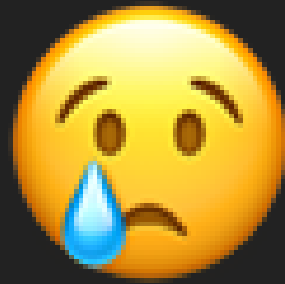
P2216 FALLOUT

P2216 FALLOUT

Boost.Locale

P2216 FALLOUT

~~Boost.Locale~~



WHAT NOW?

```
1 unsigned amount = 3;
2 format(translate("she got an apple", "she got {} apples"), plural{ amount });
3
4 // "sie bekam 3 Äpfel"      (de)
5 // "dostala 3 jablka"      (cs)
6 // "dobila je 3 jabolke"   (si)
```

WHAT NOW?

```
1 unsigned amount = 3;
2 format(translate("she got an apple", "she got {} apples"), plural{ amount });
3
4 // "sie bekam 3 Äpfel"      (de)
5 // "dostala 3 jablka"      (cs)
6 // "dobila je 3 jabolke"   (si)
```



```
1 namespace std {
2
3 struct format_string {
4     consteval format_string(const convertible_to<string_view> & Str) : Str_(Str) { ... }
5
6     constexpr std::string_view get() const noexcept { return Str_; }
7 };
8
9 }
```

WHAT NOW?

```
1 unsigned amount = 3;
2 format(translate("she got an apple", "she got {} apples"), plural{ amount });
3
4 // "sie bekam 3 Äpfel"      (de)
5 // "dostala 3 jablka"      (cs)
6 // "dobila je 3 jabolke"   (si)
```



```
1 namespace std {
2
3 struct format_string {
4     consteval format_string(const convertible_to<string_view> & Str) : Str_(Str) { ... }
5
6     constexpr std::string_view get() const noexcept { return Str_; }
7 };
8
9 }
```

MO' OVERLOADS



```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     if constexpr (sizeof...(Args) > 0) {
4         const auto Quantity = XFmt.Quantity( ... something with Args );
5         return std::vformat(XFmt.get(Quantity),
6                             make_format_args(
7                                 wrapped<std::remove_cvref_t<Types>>::translate(Args)...
8                                 ));
9     } else {
10         return std::vformat(XFmt.get(), {});
11     }
12 }
```

MO' OVERLOADS



```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     if constexpr (sizeof...(Args) > 0) {
4         const auto Quantity = XFmt.Quantity( ... something with Args );
5         return std::vformat(XFmt.get(Quantity),
6                             make_format_args(
7                                 wrapped<std::remove_cvref_t<Types>>::translate(Args)...
8                                 ));
9     } else {
10         return std::vformat(XFmt.get(), {});
11     }
12 }
```

FORMAT_STRING_TRANSLATOR

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator {
6     using base = format_string<Types...>;
7
8     consteval format_string_translator(const tTranslate & Tr)           // almost all work is done here
9     : basic_translator(Tr) {
10         base{ Tr.Singular() }, base{ Tr.Plural() }, ...;
11     }
12 };
```

FORMAT_STRING_TRANSLATOR

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator {
6     using base = format_string<Types...>;
7
8     consteval format_string_translator(const tTranslate & Tr)           // almost all work is done here
9     : basic_translator(Tr) {
10         base{ Tr.Singular() }, base{ Tr.Plural() }, ...;
11     }
12 };
```

FORMAT_STRING_TRANSLATOR

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator {
6     using base = format_string<Types...>;
7
8     static constexpr auto numPluralArguments = (isMarkedAsPlural<Types> + ... + 0);
9     static_assert(numPluralArguments <= 1, "Oops, more than one plural argument was found!");
10
11     consteval format_string_translator(const tTranslate & Tr)          // almost all work is done here
12     : basic_translator(Tr) {
13         base{ Tr.Singular() }, base{ Tr.Plural() }, checkPlural(Tr.lenPlural_, numPluralArguments);
14     }
15 };
16
17 consteval void checkPlural(std::size_t gotPluralFormat, std::size_t havePluralArguments) {
18     if (gotPluralFormat and not havePluralArguments)
19         throw "Sorry, a plural format string is present, but no plural argument was found!";
20     if (havePluralArguments and not gotPluralFormat)
21         throw "Sorry, a plural argument was found, but no plural format string is present!";
22 }
23
24 template <typename T>
25 constexpr bool isMarkedAsPlural = std::is_same_v<plural, T>;
```

BASIC_TRANSLATOR

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator; // knows argument types, does syntax checking
6
7 struct basic_translator { // knows only string digest, and how to translate from actual cardinal
8     consteval basic_translator(const tTranslate & Tr)
9     : Translator_{ markPluralsPresent(Tr.Digest_, Tr.lenPlural_ > 0) } {}
10
11 private:
12     tBaseTranslate Translator_; // strips off all strings, knows only the digest
13 };
```

BASIC_TRANSLATOR

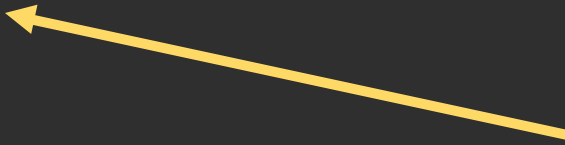
```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator; // knows argument types, does syntax checking
6
7 struct basic_translator { // knows only string digest, and how to translate from actual cardinal
8     consteval basic_translator(const tTranslate & Tr)
9     : Translator_{ markPluralsPresent(Tr.Digest_, Tr.lenPlural_ > 0) } {}
10
11     std::string_view get(plural::type N) const noexcept {
12         return havePlurals(Translator_.Digest_) ? Translator_.multiple(N) : Translator_.single();
13     }
14
15     std::string_view get() const noexcept { return Translator_.single(); }
16
17 private:
18     tBaseTranslate Translator_; // strips off all strings, knows only the digest
19 };
20
21 constexpr bool havePlurals(const uint64_t Digest) { ... }
22
23 consteval uint64_t markPluralsPresent(const uint64_t Digest, const bool havePlural) { ... }
```

BASETRANSLATE

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator; // knows argument types, does syntax checking
6
7 struct basic_translator; // knows only string digest, and how to translate from actual cardinal
8
9 struct tBaseTranslate {
10
11     std::uint64_t Digest_; // constructed by frontend at compile-time
12 };
```

BASETRANSLATE

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator; // knows argument types, does syntax checking
6
7 struct basic_translator; // knows only string digest, and how to translate from actual cardinal
8
9 struct tBaseTranslate {
10
11     std::uint64_t Digest_; // constructed by frontend at compile-time
12 };
```



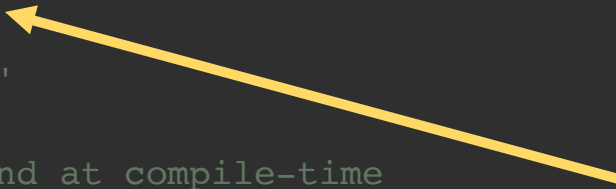
the only
data member

BASETRANSLATE

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator; // knows argument types, does syntax checking
6
7 struct basic_translator; // knows only string digest, and how to translate from actual cardinal
8
9 struct tBaseTranslate { // perform translation from compile-time digest
10     constexpr auto single() const noexcept -> std::string_view {
11         const auto & Maps = ... // from backend;
12         return lookup(Maps, Digest_); // middle: wed frontend and backend □
13     }
14
15     constexpr auto multiple(plural::type N) const noexcept -> std::string_view {
16         const auto & Maps = ... // from backend;
17         return lookup(Maps, Digest_, std::uint64_t{ N }); // middle: wed frontend and backend □
18     }
19
20     ... more overloads of 'single()' and 'multiple(N)'
21
22     std::uint64_t Digest_; // constructed by frontend at compile-time
23 };
```

BASETRANSLATE

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...>, Types &&...) -> std::string;
3
4 template <typename... Types>
5 struct format_string_translator : basic_translator; // knows argument types, does syntax checking
6
7 struct basic_translator; // knows only string digest, and how to translate from actual cardinal
8
9 struct tBaseTranslate { // perform translation from compile-time digest
10     constexpr auto single() const noexcept -> std::string_view {
11         const auto & Maps = ... // from backend;
12         return lookup(Maps, Digest_); // middle: wed frontend and backend □
13     }
14
15     constexpr auto multiple(plural::type N) const noexcept -> std::string_view {
16         const auto & Maps = ... // from backend;
17         return lookup(Maps, Digest_, std::uint64_t{ N }); // middle: wed frontend and backend □
18     }
19
20     ... more overloads of 'single()' and 'multiple(N)'
21
22     std::uint64_t Digest_; // constructed by frontend at compile-time
23 };
```



the only
runtime code

3 QUESTIONS

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     const auto Quantity = XFmt.Quantity( ... something with Args );
4     return std::vformat(XFmt.get(Quantity),
5                         make_format_args(
6                             ... somehow translate Args when necessary
7                             );
8 }
```

3 QUESTIONS

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     const auto Quantity = XFmt.Quantity( ... something with Args );
4     return std::vformat(XFmt.get(Quantity),
5                         make_format_args(
6                             ... somehow translate Args when necessary
7                             );
8 }
```

1 which argument holds the cardinal that determines the language form?

3 QUESTIONS

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     const auto Quantity = XFmt.Quantity( ... something with Args );
4     return std::vformat(XFmt.get(Quantity),
5                         make_format_args(
6                             ... somehow translate Args when necessary
7                             );
8 }
```

- 1 which argument holds the cardinal that determines the language form?
- 2 how to access its current value?

3 QUESTIONS

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     const auto Quantity = XFmt.Quantity( ... something with Args ); 1, 2
4     return std::vformat(XFmt.get(Quantity),
5                         make_format_args(
6                             ... somehow translate Args when necessary 3
7                             );
8 }
```

- 1 which argument holds the cardinal that determines the language form?
- 2 how to access its current value?
- 3 how to figure out how a given argument is translated? Or is it at all?

3 QUESTIONS

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     const auto Quantity = XFmt.Quantity( ... something with Args ); 1, 2
4     return std::vformat(XFmt.get(Quantity),
5                         make_format_args(
6                             wrapped<std::remove_cvref_t<Types>>::translate(Args)... ) 3
7                         );
8 }
```

- 1 which argument holds the cardinal that determines the language form?
- 2 how to access its current value?
- 3 how to figure out how a given argument is translated? Or is it at all?

P2663 (PACK INDEXING) + P2996 (REFLECTION)

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     constexpr auto Index = std::ranges::find_if({ Args^... }, isPluralType) - &Args...[0]^;
4     return std::vformat(XFmt.get(Args...[Index]), 2 1
5         make_format_args( [... something with Args^... ...]));
6 }
```

3

- 1 which argument holds the cardinal that determines the language form?
- 2 how to access its corrent value?
- 3 how to figure out how a given type is translated? Or is it at all?

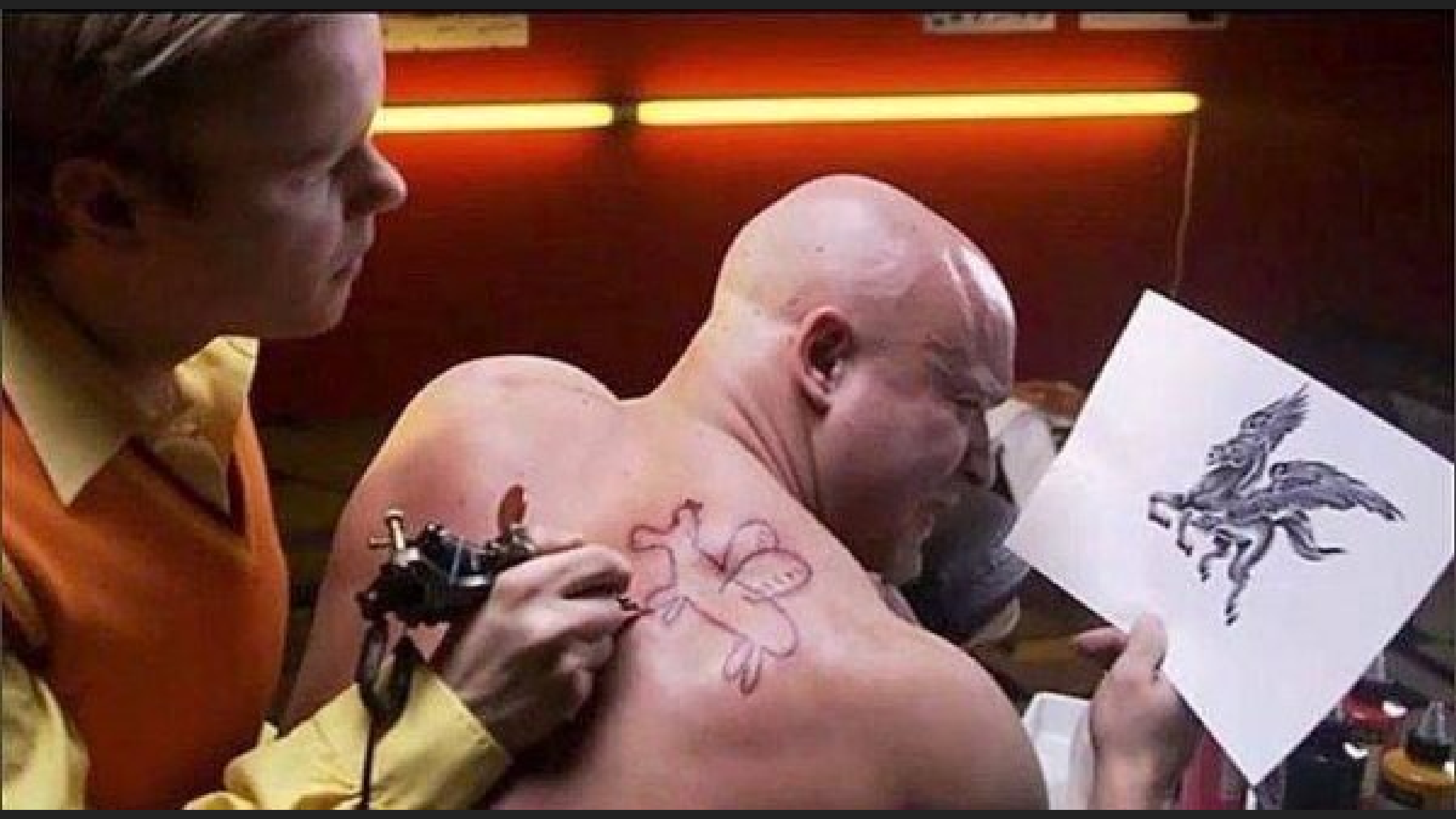
P2663 (PACK INDEXING) + P2996 (REFLECTION)

```
1 template <typename... Types>
2 auto format(const format_string_translator<Types...> XFmt, Types &&... Args) -> std::string {
3     constexpr auto Index = std::ranges::find_if({ Args^... }, isPluralType) - &Args...[0]^;
4     return std::vformat(XFmt.get(Args...[Index]), 2 1
5         make_format_args( [... something with Args^... ...]));
6 }
```

3

- 1 which argument holds the cardinal that determines the language form?
- 2 how to access its corrent value?
- 3 how to figure out how a given type is translated? Or is it at all?

Unfortunately, this is 2024. C++26 is not a thing yet 🤯



HETEROGENEOUS SEQUENCES

HETEROGENEOUS SEQUENCES



Folds!



HETEROGENEOUS SEQUENCES

A tutorial
on the universality
and expressiveness of fold

Graham Hutton, Journal of Functional Programming, 1999

SOME EXAMPLES

SOME EXAMPLES

homogeneous

e.g. vector **v** of **values**

heterogeneous

e.g. type list **<typename... Ts>**

SOME EXAMPLES

homogeneous
e.g. vector **v** of **values**

count(v) → size_t
count_if(v, pred) → size_t
count_if(v, pred, proj) → size_t
for_each(v, func) → void
for_each(v, func, proj) → void
all(v, pred) → bool
any(v, pred) → bool
none(v, pred) → bool

heterogeneous
e.g. type list <**typename**... **Ts**>

size...(Ts) → size_t
(pred<Ts> + ... + 0) → size_t
(pred<proj<Ts>> + ... + 0) → size_t
(func<Ts>, ...) → void
(func<proj<Ts>>, ...) → void
(pred<Ts> and ...) → bool
(pred<Ts> or ...) → bool
((not pred<Ts>) and ...) → bool

1 FIND_IF

Something like 'std::ranges::find_if' on a type list

1 FIND_IF

Something like 'std::ranges::find_if' on a type list

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>                                // the predicate, T -> bool
4 struct isMarkedAsPlural {                             // needs to be wrapped in a class for reasons
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto numPluralArguments = (isMarkedAsPlural<Types>{} + ... + 0);
11     static_assert(numPluralArguments <= 1, "Oops, more than one plural argument was found!");
12
13     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
14     static_assert(numPluralArguments > 0 ? PluralIndex < sizeof...(Types) : PluralIndex == noIndex);
15 };
```

1 FIND_IF

Something like 'std::ranges::find_if' on a type list

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>                                // the predicate,  T -> bool
4 struct isMarkedAsPlural {                             // needs to be wrapped in a class for reasons
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto numPluralArguments = (isMarkedAsPlural<Types>{} + ... + 0);
11     static_assert(numPluralArguments <= 1, "Oops, more than one plural argument was found!");
12
13     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
14     static_assert(numPluralArguments > 0 ? PluralIndex < sizeof...(Types) : PluralIndex == noIndex);
15 };
```

1 FIND_IF

Something like 'std::ranges::find_if' on a type list

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>                                // the predicate, T -> bool
4 struct isMarkedAsPlural {                             // needs to be wrapped in a class for reasons
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto numPluralArguments = (isMarkedAsPlural<Types>{} + ... + 0);
11     static_assert(numPluralArguments <= 1, "Oops, more than one plural argument was found!");
12
13     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
14     static_assert(numPluralArguments > 0 ? PluralIndex < sizeof...(Types) : PluralIndex == noIndex);
15 };
```



1 FIND_IF

Something like 'std::ranges::find_if' on a type list

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>                                // the predicate,  T -> bool
4 struct isMarkedAsPlural {                            // needs to be wrapped in a class for reasons
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto numPluralArguments = (isMarkedAsPlural<Types>{} + ... + 0);
11     static_assert(numPluralArguments <= 1, "Oops, more than one plural argument was found!");
12
13     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
14     static_assert(numPluralArguments > 0 ? PluralIndex < sizeof...(Types) : PluralIndex == noIndex);
15 };
```



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex;
11
12     return Result;
13 }
14
15 static_assert(findFirstIndex() == noIndex);
```



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex;
11
12     return Result;
13 }
14
15 static_assert(findFirstIndex() == noIndex);
```



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex;
11
12     return Result;
13 }
14
15 static_assert(findFirstIndex() == noIndex);
```



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11
12     return Result - 1;
13 }
14
15 static_assert(findFirstIndex() == noIndex);
```



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <template <typename T> typename Predicate, typename... Types>
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11
12     return (false or ... or Predicate<Types>{}),
13           Result - 1;
14 }
15
16 static_assert(findFirstIndex<isMarkedAsPlural>() == noIndex); ✓
17 static_assert(findFirstIndex<isMarkedAsPlural, void>() == noIndex); ✓
18 static_assert(findFirstIndex<isMarkedAsPlural, plural, void>() == noIndex); ✗
19 static_assert(findFirstIndex<isMarkedAsPlural, void, plural>() == noIndex); ✗
```

FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <template <typename T> typename Predicate, typename... Types>
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11
12     return (false or ... or Predicate<Types>{}),
13           Result - 1;
14 }
15
16 static_assert(findFirstIndex<isMarkedAsPlural>() == noIndex);
17 static_assert(findFirstIndex<isMarkedAsPlural, void>() == noIndex);
18 static_assert(findFirstIndex<isMarkedAsPlural, plural, void>() == noIndex);
19 static_assert(findFirstIndex<isMarkedAsPlural, void, plural>() == noIndex);
```

must be a type because of
<http://eel.is/c++draft/temp.arg.template>



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <template <typename T> typename Predicate, typename... Types>
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11
12     return (false or ... or Predicate<Types>{}),
13           Result - 1;
14 }
15
16 static_assert(findFirstIndex<isMarkedAsPlural>() == noIndex);
17 static_assert(findFirstIndex<isMarkedAsPlural, void>() == noIndex);
18 static_assert(findFirstIndex<isMarkedAsPlural, plural, void>() == noIndex);
19 static_assert(findFirstIndex<isMarkedAsPlural, void, plural>() == noIndex);
```

must be a type because of
<http://eel.is/c++draft/temp.arg.template>



FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <template <typename T> typename Predicate, typename... Types>
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11
12     return (false or ... or (Predicate<Types>{} ? true : false)),
13           Result - 1;
14 }
15
16 static_assert(findFirstIndex<isMarkedAsPlural>() == noIndex); ✓
17 static_assert(findFirstIndex<isMarkedAsPlural, void>() == noIndex); ✓
18 static_assert(findFirstIndex<isMarkedAsPlural, plural, void>() == noIndex); ✗
19 static_assert(findFirstIndex<isMarkedAsPlural, void, plural>() == noIndex); ✗
```

FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <template <typename T> typename Predicate, typename... Types>
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11     std::size_t Index  = 0;
12
13     return (false or ... or (++Index, Predicate<Types>{} ? true : false)),
14           Result - 1;
15 }
16
17 static_assert(findFirstIndex<isMarkedAsPlural>() == noIndex);      ✓
18 static_assert(findFirstIndex<isMarkedAsPlural, void>() == noIndex); ✓
19 static_assert(findFirstIndex<isMarkedAsPlural, plural, void>() == noIndex); ✗
20 static_assert(findFirstIndex<isMarkedAsPlural, void, plural>() == noIndex); ✗
```

FINDFIRSTINDEX

```
1 static constexpr auto noIndex = std::size_t{ 0 } - 1;
2
3 template <typename T>
4 struct isMarkedAsPlural {
5     constexpr operator bool() const noexcept { return std::is_same_v<plural, T>; }
6 };
7
8 template <template <typename T> typename Predicate, typename... Types>
9 constexpr std::size_t findFirstIndex() {
10     std::size_t Result = noIndex + 1;
11     std::size_t Index  = 0;
12
13     return (false or ... or (++Index, Predicate<Types>{} ? static_cast<bool>(Result = Index) : false)),
14           Result - 1;
15 }
16
17 static_assert(findFirstIndex<isMarkedAsPlural>() == noIndex); ✓
18 static_assert(findFirstIndex<isMarkedAsPlural, void>() == noIndex); ✓
19 static_assert(findFirstIndex<isMarkedAsPlural, plural, void>() == 0); ✓
20 static_assert(findFirstIndex<isMarkedAsPlural, void, plural>() == 1); ✓
```



2 INDEX ARGUMENT

```
1 template <typename... Types>
2 std::string format(const format_string_translator<Types...> XFmt, Types &&... Args) {
3     const auto Quantity = XFmt.Quantity(
4         { reinterpret_cast<uintptr_t>(std::addressof(Args))... }
5     );
6 }
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
11
12     static plural::type Quantity(const uintptr_t (&pArgs)[]) noexcept {
13         if constexpr (PluralIndex != noIndex)
14             return *std::bit_cast<const plural *>(pArgs[PluralIndex]);
15         else
16             return 1;
17     }
18 };
```

2 INDEX ARGUMENT

```
1 template <typename... Types>
2 std::string format(const format_string_translator<Types...> XFmt, Types &&... Args) {
3     const auto Quantity = XFmt.Quantity(
4         { reinterpret_cast<uintptr_t>(std::addressof(Args))... }
5     );
6 }
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
11
12     static plural::type Quantity(const uintptr_t (&pArgs)[]) noexcept {
13         if constexpr (PluralIndex != noIndex)
14             return *std::bit_cast<const plural *>(pArgs[PluralIndex]);
15         else
16             return 1;
17     }
18 };
```

2 INDEX ARGUMENT

```
1 template <typename... Types>
2 std::string format(const format_string_translator<Types...> XFmt, Types &&... Args) {
3     const auto Quantity = XFmt.Quantity(
4         { reinterpret_cast<uintptr_t>(std::addressof(Args))... }
5     );
6 }
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
11
12     static plural::type Quantity(const uintptr_t (&pArgs)[]) noexcept {
13         if constexpr (PluralIndex != noIndex)
14             return *std::bit_cast<const plural *>(pArgs[PluralIndex]);
15         else
16             return 1;
17     }
18 };
```

2 INDEX ARGUMENT

```
1 template <typename... Types>
2 std::string format(const format_string_translator<Types...> XFmt, Types &&... Args) {
3     const auto Quantity = XFmt.Quantity(
4         { reinterpret_cast<uintptr_t>(std::addressof(Args))... }
5     );
6 }
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
11
12     static plural::type Quantity(const uintptr_t (&pArgs)[]) noexcept {
13         if constexpr (PluralIndex != noIndex)
14             return *std::bit_cast<const plural *>(pArgs[PluralIndex]);
15         else
16             return 1;
17     }
18 };
```

2 INDEX ARGUMENT

```
1 template <typename... Types>
2 std::string format(const format_string_translator<Types...> XFmt, Types &&... Args) {
3     const auto Quantity = XFmt.Quantity(
4         { reinterpret_cast<uintptr_t>(std::addressof(Args))... }
5     );
6 }
7
8 template <typename... Types>
9 struct format_string_translator {
10     static constexpr auto PluralIndex = findFirstIndex<isMarkedAsPlural, Types...>();
11
12     static plural::type Quantity(const uintptr_t (&pArgs)[]) noexcept {
13         if constexpr (PluralIndex != noIndex) {
14             return *std::bit_cast<const plural *>(pArgs[PluralIndex]);
15         }
16         else {
17             return 1;
18         }
19     };
20 }
```

This is valid code, no undefined behaviour here! 😊

3 TYPE-BASED SELECTION

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

3 TYPE-BASED SELECTION

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

3 TYPE-BASED SELECTION

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

3 TYPE-BASED SELECTION

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

3 TYPE-BASED SELECTION

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

3 TYPE-BASED SELECTION

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

3 TYPE-BASED SELECTION

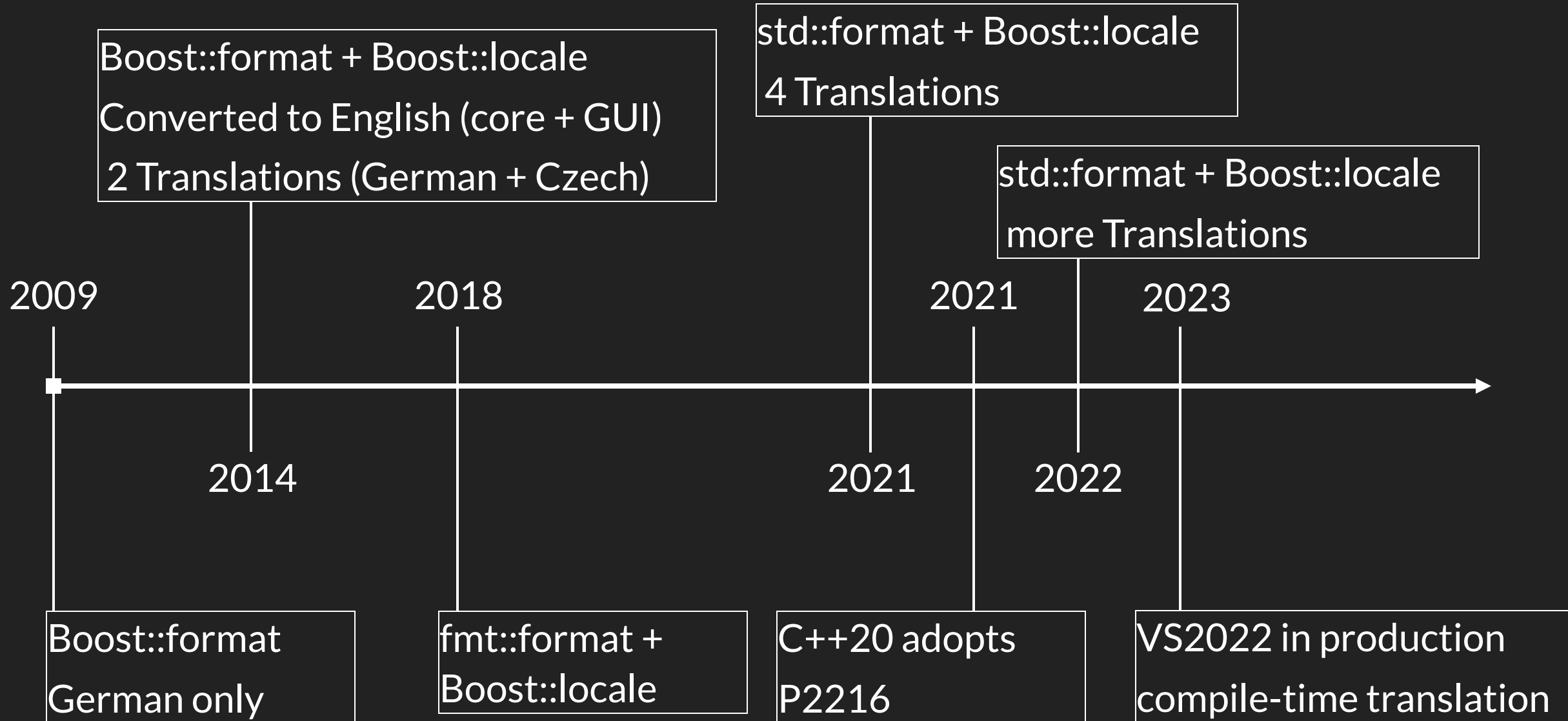
a.k.a. pattern-matching (P2688)

```
1 template <typename T>
2 struct wrapped {
3     using type = T;
4     static constexpr const type & translate(const T & t) { return t; }
5 };
6
7 template <typename Char>
8 struct wrapped<tTranslate<Char>> {
9     using type = std::basic_string_view<Char>;
10    static constexpr type translate(const tBaseTranslate<Char> & t) { return t.single(); }
11 };
12
13 template <>
14 struct wrapped<plural> {
15     using type = plural::type;
16     static constexpr type translate(const plural & t) { return t; }
17 };
```

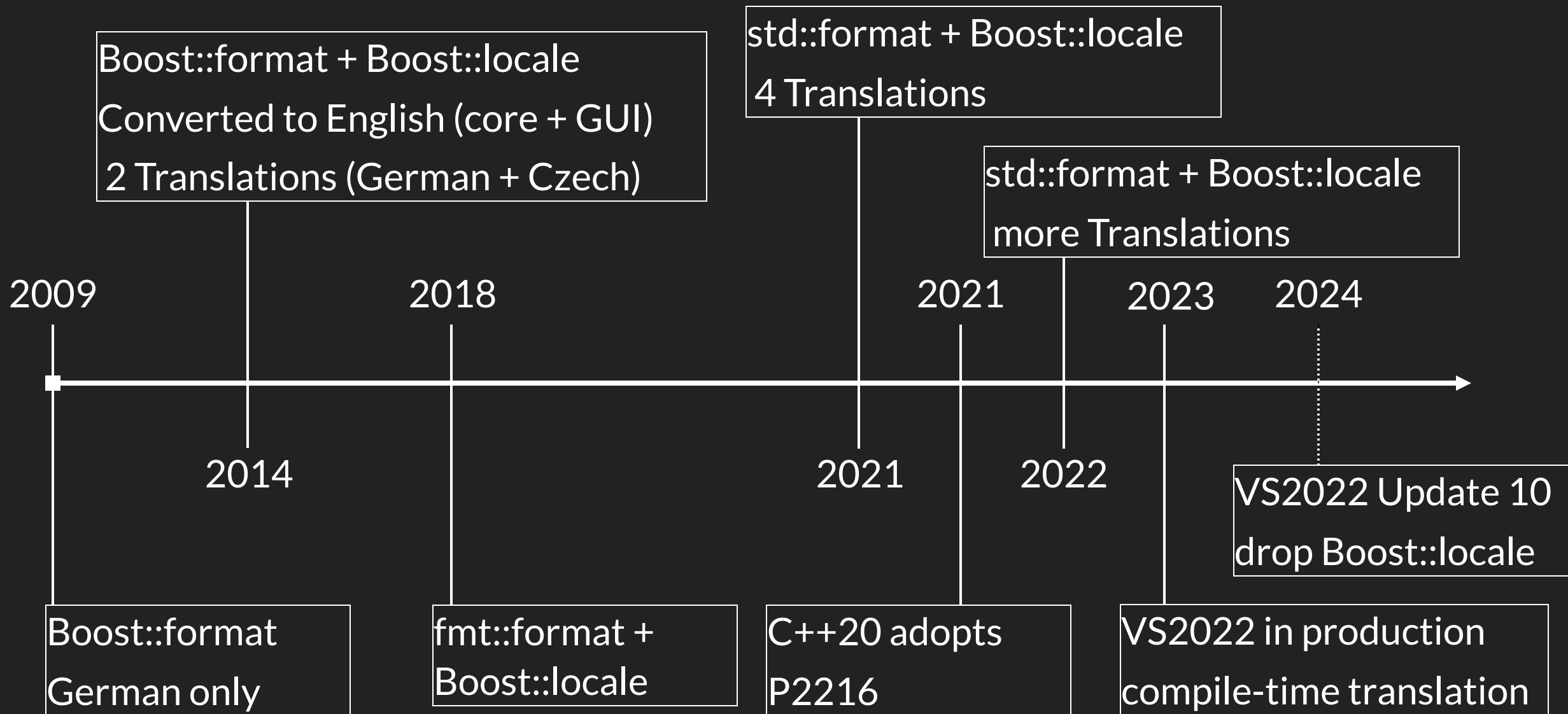
an
experiment



TIMELINE



TIMELINE



INTEGRATION TEST

```
constexpr auto Test(  
    const auto Source,  
    const uint64_t Cardinal    = 1'000'000,  
    const wstring_view Result = L"Language Form 3") {  
    const auto Tr = tTranslator(TranslationDomain("Integration"))  
        .load(LanguageId("br"), Source);  
    return Result ==  
        translate("IntegrationContext", L"Singular", L"Plural")  
        ._(Cardinal, Tr);  
}
```

INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal    = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

Compiler

Build system

INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal    = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

Compiler

Build system

INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal    = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

Build system

.pot

'gettext'
Tools

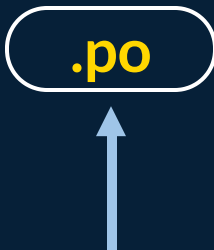
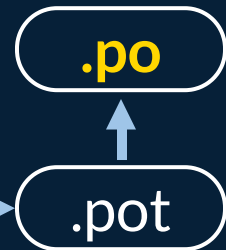
Compiler

INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal    = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13                .load(LanguageId("br"), Source);
14    return Result ==
15           translate("IntegrationContext", L"Singular", L"Plural")
16           ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

Compiler

Build system



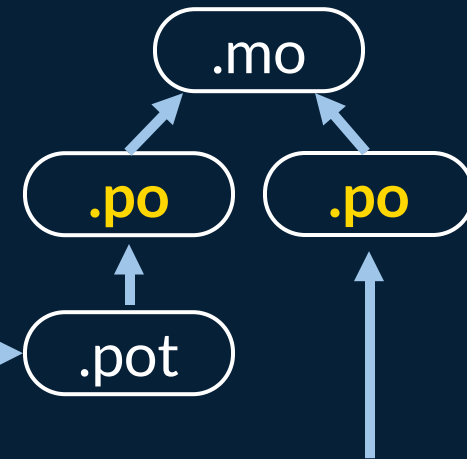
'gettext'
Tools



INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10     const uint64_t Cardinal    = 1'000'000,
11     const wstring_view Result = L"Language Form 3") {
12     const auto Tr = tTranslator(TranslationDomain("Integration"))
13         .load(LanguageId("br"), Source);
14     return Result ==
15         translate("IntegrationContext", L"Singular", L"Plural")
16             ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

Build system



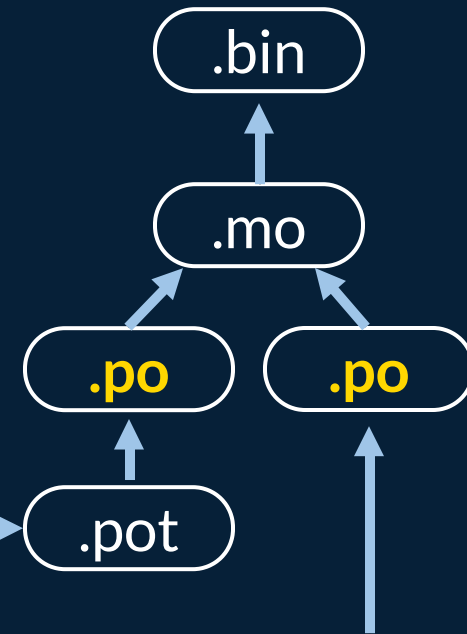
'gettext'
Tools



INTEGRATION TEST

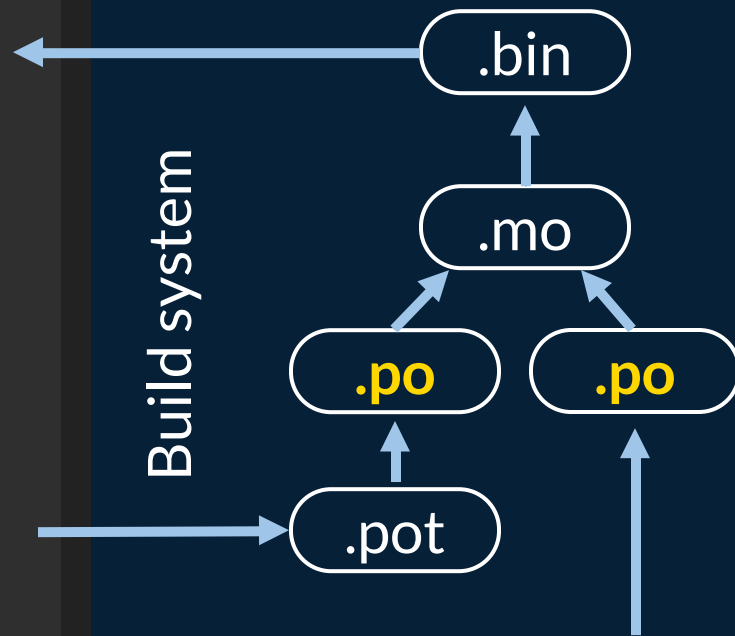
```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10     const uint64_t Cardinal    = 1'000'000,
11     const wstring_view Result = L"Language Form 3") {
12     const auto Tr = tTranslator(TranslationDomain("Integration"))
13         .load(LanguageId("br"), Source);
14     return Result ==
15         translate("IntegrationContext", L"Singular", L"Plural")
16             ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

Build system



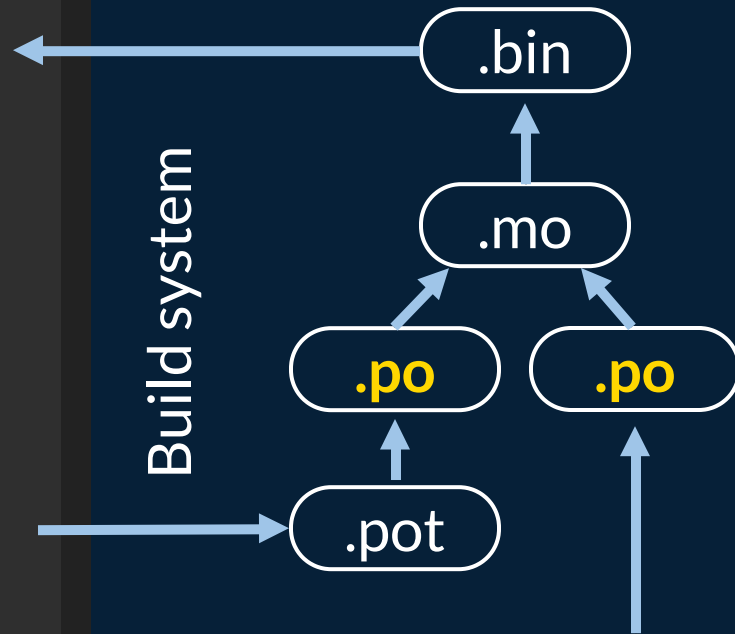
INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10     const uint64_t Cardinal    = 1'000'000,
11     const wstring_view Result = L"Language Form 3") {
12     const auto Tr = tTranslator(TranslationDomain("Integration"))
13         .load(LanguageId("br"), Source);
14     return Result ==
15         translate("IntegrationContext", L"Singular", L"Plural")
16             ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```



INTEGRATION TEST

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10     const uint64_t Cardinal    = 1'000'000,
11     const wstring_view Result = L"Language Form 3") {
12     const auto Tr = tTranslator(TranslationDomain("Integration"))
13         .load(LanguageId("br"), Source);
14     return Result ==
15         translate("IntegrationContext", L"Singular", L"Plural")
16             ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```



INTEGRATION TEST

Compiler

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal    = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

br.po

```
"Project-Id-Version: Integration"
"Language: br"
"Plural-Forms: nplurals=5; plural=
(n%10==1 && n%100!=11 &&
n%100!=71 && n%100!=91 ? 0 :
n%10==2 && n%100!=12 &&
n%100!=72 && n%100!=92 ? 1 :
((n%10>=3 && n%10<=4) ||
n%10==9) &&
(n%100<10 || n%100>19) &&
(n%100<70 || n%100>79) &&
(n%100<90 || n%100>99) ? 2 :
n!=0 && n%1000000==0 ? 3 :
4);"
```

...

```
#: main.cpp
msgctxt "IntegrationContext"
msgid "Singular"
msgid_plural "Plural"
msgstr[0] "Language Form 0"
msgstr[1] "Language Form 1"
msgstr[2] "Language Form 2"
msgstr[3] "Language Form 3"
msgstr[4] "Language Form 4"
```

...

INTEGRATION TEST

Compiler

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin" // bring in the Breton translations (~ 500)
6 ); // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

↪ 6CC901FEE4B74CB5

br.po

```
"Project-Id-Version: Integration"
"Language: br"
"Plural-Forms: nplurals=5; plural=
(n%10==1 && n%100!=11 &&
n%100!=71 && n%100!=91 ? 0 :
n%10==2 && n%100!=12 &&
n%100!=72 && n%100!=92 ? 1 :
((n%10>=3 && n%10<=4) ||
n%10==9) &&
(n%100<10 || n%100>19) &&
(n%100<70 || n%100>79) &&
(n%100<90 || n%100>99) ? 2 :
n!=0 && n%1000000==0 ? 3 :
4);"

...

#: main.cpp
msgctxt "IntegrationContext"
msgid "Singular"
msgid_plural "Plural"
msgstr[0] "Language Form 0"
msgstr[1] "Language Form 1"
msgstr[2] "Language Form 2"
msgstr[3] "Language Form 3"
msgstr[4] "Language Form 4"

...
```

INTEGRATION TEST

Compiler

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin" // bring in the Breton translations (~ 500)
6 ); // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

br.po

```
"Project-Id-Version: Integration"
"Language: br"
"Plural-Forms: nplurals=5; plural=
(n%10==1 && n%100!=11 &&
n%100!=71 && n%100!=91 ? 0 :
n%10==2 && n%100!=12 &&
n%100!=72 && n%100!=92 ? 1 :
((n%10>=3 && n%10<=4) ||
n%10==9) &&
(n%100<10 || n%100>19) &&
(n%100<70 || n%100>79) &&
(n%100<90 || n%100>99) ? 2 :
n!=0 && n%1000000==0 ? 3 :
4);"
```

...

```
#: main.cpp
msgctxt "IntegrationContext"
msgid "Singular"
msgid_plural "Plural"
msgstr[0] "Language Form 0"
msgstr[1] "Language Form 1"
msgstr[2] "Language Form 2"
msgstr[3] "Language Form 3"
msgstr[4] "Language Form 4"
```

...

INTEGRATION TEST

Compiler

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin" // bring in the Breton translations (~ 500)
6 ); // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10    const uint64_t Cardinal = 1'000'000,
11    const wstring_view Result = L"Language Form 3") {
12    const auto Tr = tTranslator(TranslationDomain("Integration"))
13        .load(LanguageId("br"), Source);
14    return Result ==
15        translate("IntegrationContext", L"Singular", L"Plural")
16        ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

br.po

```
"Project-Id-Version: Integration"
"Language: br"
"Plural-Forms: nplurals=5; plural=
(n%10==1 && n%100!=11 &&
n%100!=71 && n%100!=91 ? 0 :
n%10==2 && n%100!=12 &&
n%100!=72 && n%100!=92 ? 1 :
((n%10>=3 && n%10<=4) ||
n%10==9) &&
(n%100<10 || n%100>19) &&
(n%100<70 || n%100>79) &&
(n%100<90 || n%100>99) ? 2 :
n!=0 && n%1000000==0 ? 3 :
4);"
```

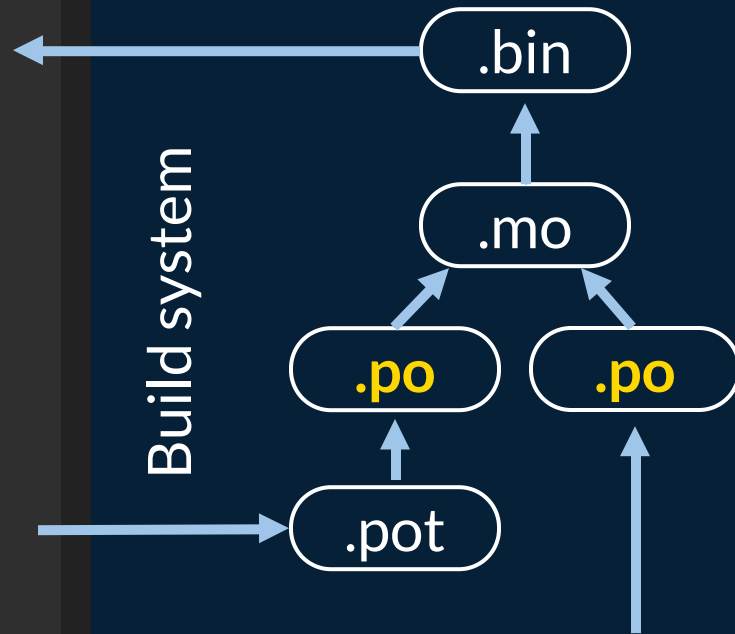
...

```
#: main.cpp
msgctxt "IntegrationContext"
msgid "Singular"
msgid_plural "Plural"
msgstr[0] "Language Form 0"
msgstr[1] "Language Form 1"
msgstr[2] "Language Form 2"
msgstr[3] "Language Form 3"
msgstr[4] "Language Form 4"
```

...

TEST AT COMPILETIME

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 static constexpr auto BretonConst = utility::embed(
5     #include "br.bin"    // bring in the Breton translations (~ 500)
6 );                      // from the compiled .mo
7
8 constexpr auto Test(
9     const auto Source,
10     const uint64_t Cardinal    = 1'000'000,
11     const wstring_view Result = L"Language Form 3") {
12     const auto Tr = tTranslator(TranslationDomain("Integration"))
13         .load(LanguageId("br"), Source);
14     return Result ==
15         translate("IntegrationContext", L"Singular", L"Plural")
16             ._(Cardinal, Tr);
17 }
18
19 static_assert(Test(BretonConst)); // a single constant evaluation!
20
21 int main() {
22     // auto BretonFile = "br.mo";
23     // assert(Test(BretonFile));
24 }
```

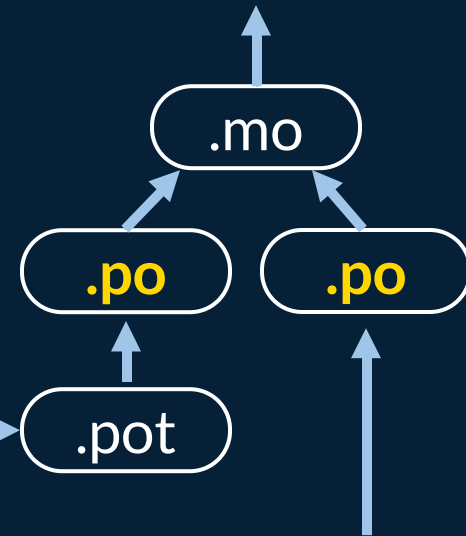


TEST AT RUNTIME

```
1 import Translate; // library implementation of the 'gettext' facilities
2 import utility;
3
4 // static constexpr auto BretonConst = utility::embed(
5 //     #include "br.bin"
6 // );
7
8 constexpr auto Test(
9     const auto Source,
10     const uint64_t Cardinal = 1'000'000,
11     const wstring_view Result = L"Language Form 3") {
12     const auto Tr = tTranslator(TranslationDomain("Integration"))
13         .load(LanguageId("br"), Source);
14     return Result ==
15         translate("IntegrationContext", L"Singular", L"Plural")
16             ._(Cardinal, Tr);
17 }
18
19 // static_assert(Test(BretonConst));
20
21 int main() {
22     auto BretonFile = "br.mo";
23     assert(Test(BretonFile)); // map or load .mo
24 }
```

Build system

filesystem



move on



RESOURCES

- Living, up-to-date C++ standard (currently at C++26)
 - The Journal of Functional Programming, Cambridge University Press
 - GNU gettext utilities
 - Unicode CLDR
-
- **Library code** github.com/DanielaE/t.b.d.

Contact

✉ dani@ngert.de

💬 @DanielaKEngert@hachyderm.io

🐙 [DanielaE](https://github.com/DanielaE)

Images: [Maria Sibylla Merian \(1705\)](#)





Ceterum censeo ABI esse frangendam